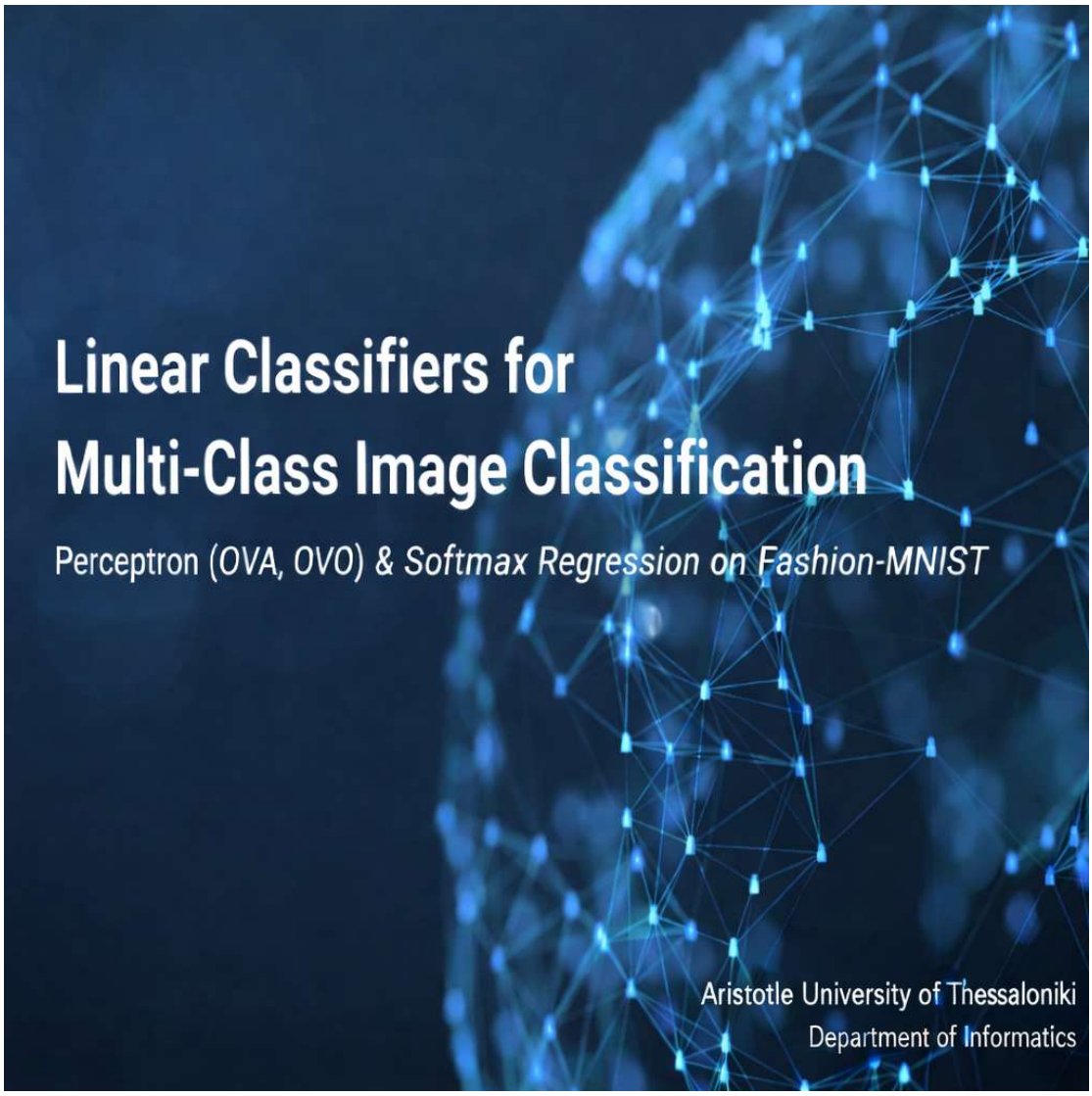
2η ΥΠΟΧΡΕΩΤΙΚΗ ΕΡΓΑΣΙΑ ΣΤΟ ΜΑΘΗΜΑ «ΑΝΑΓΝΩΡΙΣΗ ΠΡΟΤΥΠΩΝ»

Linear Classifiers for Multi-Class Image Classification
Perceptron (OVA, OVO) & Softmax Regression on Fashion-MNIST



Γρηγόρης Λιάτσος, ΑΕΜ:4540

Υπεύθυνος Καθηγητής: Αναστάσιος Τέφας

Τμήμα Πληροφορικής – ΑΠΘ: Απρίλιος 2026

Περιεχόμενα

Abstract ... 3
Εισαγωγή ... 4
1. Βάση Δεδομένων FASHION-MNIST ... 5
2. Προ-επεξεργασία ... 6
3. Θεωρητικό Υπόβαθρο ... 6
3.1 Γραμμικοί Ταξινομητές (Linear Classifiers) ... 6
3.2 Perceptron ... 7
3.2.1 Δυαδικός Perceptron ... 7
3.2.2 Perceptron One-vs-All (OvA) ... 7
3.2.3 Perceptron One-vs-One (OvO) ... 7
3.2.4 Softmax Regression ... 8
4. Ποιοτική Ανάλυση Αποτελεσμάτων ... 9
4.1 Ποιοτική Ανάλυση Αποτελεσμάτων ... 9
4.2 Ανάλυση Confusion Matrices ... 11
4.2.1 Perceptron OVA (One-vs-All) ... 11
4.2.2 Perceptron OVO (One-vs-One) ... 13
4.2.3 Softmax Regression ... 14
4.3 Συμπεράσματα από την Ποιοτική Ανάλυση ... 16
5. Αποτελέσματα ... 16
5.1 Εισαγωγή ... 16
5.2 Αποτελέσματα σε Balanced Dataset ... 16
5.3 Αποτελέσματα σε Imbalanced Dataset ... 17
5.4 Συγκριτική Ανάλυση ... 18
6. Συμπεράσματα ... 19
7. Ανάλυση Κώδικα ... 20
8. Παράρτημα – Πλήρης Κώδικας ... 41
9. Βιβλιογραφία ... 54

Abstract

Η παρούσα εργασία εξετάζει την υλοποίηση και τη συγκριτική αξιολόγηση γραμμικών ταξινομητών για προβλήματα πολυκατηγορικής ταξινόμησης εικόνων, χρησιμοποιώντας το σύνολο δεδομένων Fashion-MNIST, το οποίο αποτελεί καθιερωμένο σημείο αναφοράς (benchmark) στον τομέα της αναγνώρισης προτύπων. Συγκεκριμένα, υλοποιούνται από το μηδέν οι αλγόριθμοι Perceptron στις εκδοχές One-vs-All (OvA) και One-vs-One (OvO), καθώς και ένας γραμμικός ταξινομητής βασισμένος στη συνάρτηση Softmax.

Η μελέτη πραγματοποιείται υπό δύο διαφορετικά σενάρια εκπαίδευσης: (α) ισορροπημένο σύνολο δεδομένων με ίσο αριθμό δειγμάτων ανά κλάση και (β) μη ισορροπημένο σύνολο, στο οποίο μία κλάση υπερισχύει σημαντικά έναντι των υπολοίπων. Τα δεδομένα υποβάλλονται σε κατάλληλη προεπεξεργασία, η οποία περιλαμβάνει κανονικοποίηση, τυποποίηση (standardization) και μετατροπή των εικόνων σε διανύσματα χαρακτηριστικών, ώστε να καταστούν συμβατά με τα γραμμικά μοντέλα.

Η εκπαίδευση των μοντέλων πραγματοποιείται με επαναληπτικές μεθόδους βελτιστοποίησης, ενώ η αξιολόγηση βασίζεται σε μετρικές όπως η ακρίβεια ταξινόμησης, οι πίνακες σύγχυσης και η ανάλυση της επίδρασης των υπερπαραμέτρων (ρυθμός μάθησης, αριθμός εποχών εκπαίδευσης). Επιπλέον, εξετάζεται η επίδραση της ανισορροπίας των δεδομένων και η χρήση τεχνικών στάθμισης κλάσεων (class weighting).

Τα πειραματικά αποτελέσματα δείχνουν ότι στο ισορροπημένο σύνολο δεδομένων ο ταξινομητής Perceptron One-vs-One επιτυγχάνει την υψηλότερη απόδοση, αξιοποιώντας τον διαχωρισμό του προβλήματος σε δυαδικές ταξινομήσεις. Αντίθετα, στο μη ισορροπημένο σενάριο, ο ταξινομητής Softmax παρουσιάζει ανώτερη και πιο σταθερή απόδοση, χάρη στην ενσωμάτωση στάθμισης κλάσεων, η οποία μειώνει τη μεροληψία προς την κυρίαρχη κλάση. Οι εκδοχές του Perceptron εμφανίζουν μεγαλύτερη ευαισθησία στην ανισορροπία των δεδομένων, οδηγώντας σε αισθητή μείωση της ακρίβειας.

Συνολικά, η εργασία αναδεικνύει τα πλεονεκτήματα και τους περιορισμούς των γραμμικών ταξινομητών σε προβλήματα εικόνας. Παρότι τα μοντέλα αυτά είναι υπολογιστικά αποδοτικά και απλά στην υλοποίηση, η γραμμική φύση τους περιορίζει την ικανότητά τους να μοντελοποιούν πολύπλοκες μη γραμμικές δομές, γεγονός που επηρεάζει την απόδοσή τους σε απαιτητικά προβλήματα ταξινόμησης.

Εισαγωγή

Η Αναγνώριση Προτύπων (Pattern Recognition) αποτελεί έναν από τους βασικότερους κλάδους της επιστήμης υπολογιστών και της μηχανικής μάθησης, με κύριο στόχο την εξαγωγή χρήσιμης πληροφορίας από δεδομένα μέσω της ανίχνευσης και κατηγοριοποίησης προτύπων. Οι εφαρμογές της εκτείνονται σε ένα ευρύ φάσμα πεδίων, όπως η αναγνώριση εικόνων, η επεξεργασία φυσικής γλώσσας, η ανάλυση βιοϊατρικών δεδομένων και τα αυτόνομα συστήματα. Η συνεχής αύξηση του όγκου και της πολυπλοκότητας των δεδομένων καθιστά απαραίτητη την ανάπτυξη αποδοτικών και αξιόπιστων μεθόδων ταξινόμησης.

Στο πλαίσιο αυτό, οι γραμμικοί ταξινομητές (linear classifiers) αποτελούν μία από τις πιο θεμελιώδεις και ευρέως χρησιμοποιούμενες κατηγορίες αλγορίθμων. Η βασική τους ιδέα είναι ο διαχωρισμός των δεδομένων μέσω γραμμικών συναρτήσεων απόφασης, οι οποίες ορίζουν υπερεπίπεδα στον χώρο χαρακτηριστικών. Παρά την απλότητά τους, προσφέρουν σημαντικά πλεονεκτήματα, όπως χαμηλό υπολογιστικό κόστος, ευκολία υλοποίησης και ικανοποιητική ικανότητα γενίκευσης σε πολλές πρακτικές εφαρμογές.

Μεταξύ των σημαντικότερων γραμμικών μοντέλων συγκαταλέγεται ο αλγόριθμος Perceptron, ο οποίος αποτελεί μία από τις πρώτες μεθόδους εποπτευόμενης μάθησης και χρησιμοποιείται για την εκμάθηση γραμμικών ορίων απόφασης μέσω επαναληπτικής προσαρμογής των βαρών. Σε προβλήματα πολλαπλών κλάσεων, ο Perceptron επεκτείνεται συνήθως μέσω στρατηγικών όπως το One-vs-All (OvA) και το One-vs-One (OvO). Παράλληλα, ιδιαίτερη σημασία έχει ο ταξινομητής Softmax (Softmax Regression), ο οποίος αποτελεί γενίκευση της λογιστικής παλινδρόμησης και επιτρέπει την άμεση εκτίμηση πιθανοτήτων για κάθε κλάση.

Σκοπός της παρούσας εργασίας είναι η υλοποίηση από το μηδέν και η συγκριτική αξιολόγηση των παραπάνω γραμμικών ταξινομητών σε πρόβλημα πολυκατηγορικής ταξινόμησης εικόνων, χρησιμοποιώντας το σύνολο δεδομένων Fashion-MNIST. Η ανάλυση πραγματοποιείται υπό δύο διαφορετικά σενάρια: ένα ισορροπημένο σύνολο δεδομένων και ένα μη ισορροπημένο, προκειμένου να διερευνηθεί η επίδραση της κατανομής των δεδομένων στην απόδοση των μοντέλων, καθώς και η ανάγκη χρήσης τεχνικών αντιμετώπισης της ανισορροπίας.

Τα δεδομένα υποβάλλονται σε διαδικασία προεπεξεργασίας που περιλαμβάνει κανονικοποίηση, τυποποίηση και μετατροπή σε διανύσματα χαρακτηριστικών, ώστε να είναι κατάλληλα για χρήση από γραμμικά μοντέλα. Η αξιολόγηση βασίζεται σε μετρικές όπως η ακρίβεια ταξινόμησης, οι πίνακες σύγχυσης και η ανάλυση της επίδρασης βασικών υπερπαραμέτρων, όπως ο ρυθμός μάθησης και ο αριθμός εποχών εκπαίδευσης.

Μέσα από τη μελέτη αυτή, επιχειρείται η ανάδειξη των πλεονεκτημάτων και των περιορισμών των γραμμικών ταξινομητών σε δεδομένα εικόνας, καθώς και η κατανόηση των συνθηκών υπό τις οποίες οι συγκεκριμένες μέθοδοι αποδίδουν αποτελεσματικά, ιδιαίτερα σε περιβάλλοντα με ανισορροπία δεδομένων.

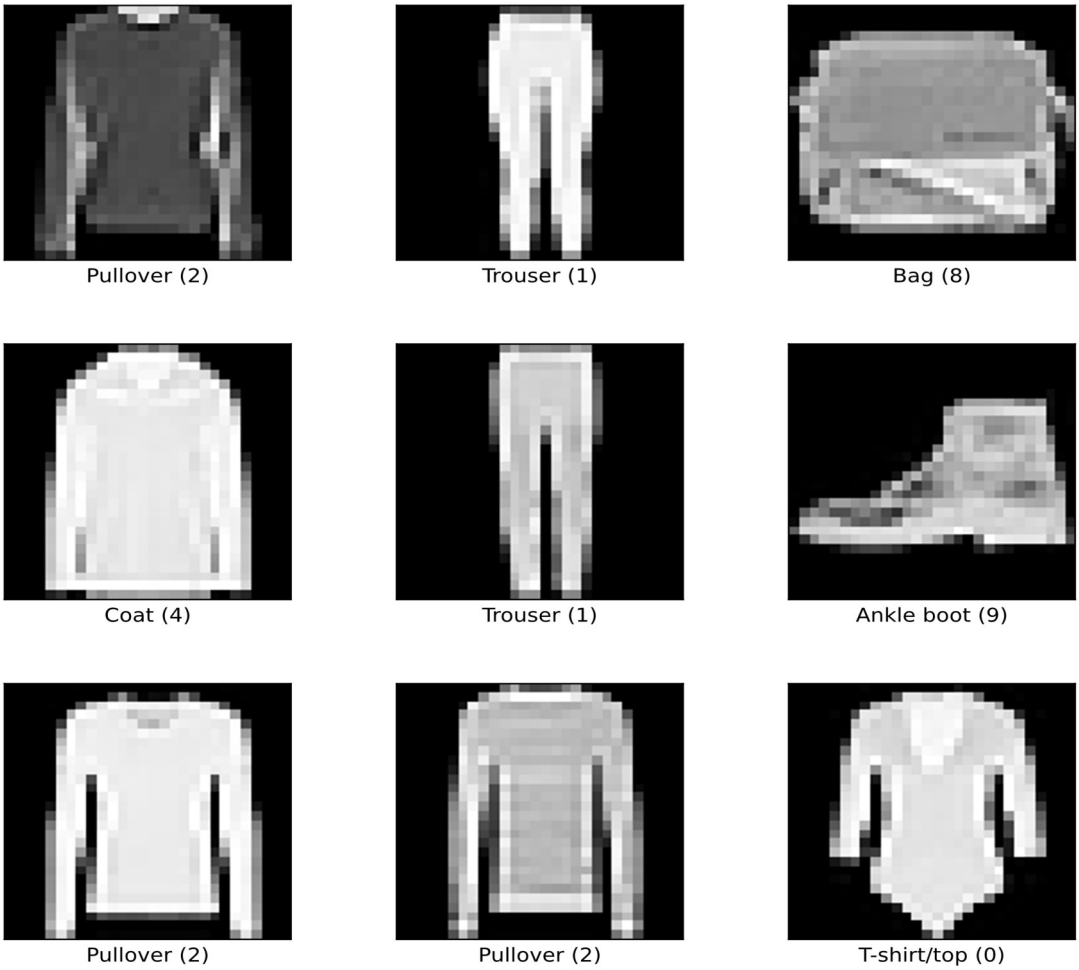
1. Βάση Δεδομένων FASHION-MNIST

Το σύνολο δεδομένων που χρησιμοποιείται στην παρούσα εργασία είναι το Fashion-MNIST, ένα ευρέως χρησιμοποιούμενο benchmark dataset για προβλήματα ταξινόμησης εικόνων. Το Fashion-MNIST δημιουργήθηκε ως μία πιο απαιτητική εναλλακτική του κλασικού MNIST, αντικαθιστώντας τα χειρόγραφα ψηφία με εικόνες ενδυμάτων.

Το dataset αποτελείται από συνολικά 70.000 εικόνες γκρι κλίμακας (grayscale), οι οποίες κατανέμονται σε 60.000 δείγματα εκπαίδευσης και 10.000 δείγματα ελέγχου (test set). Οι εικόνες έχουν διαστάσεις 28×28 pixels, γεγονός που τις καθιστά κατάλληλες για γρήγορη επεξεργασία και εφαρμογή αλγορίθμων μηχανικής μάθησης.

Κάθε εικόνα αντιστοιχεί σε μία από τις ακόλουθες 10 κατηγορίες: T-shirt/Top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag και Ankle Boot.

Η επιλογή του Fashion-MNIST είναι ιδιαίτερα κατάλληλη για την παρούσα εργασία, καθώς, παρότι πρόκειται για dataset χαμηλής διάστασης, παρουσιάζει μεγαλύτερη πολυπλοκότητα σε σχέση με το MNIST, καθιστώντας πιο απαιτητική την ταξινόμηση με γραμμικούς ταξινομητές. Αυτό επιτρέπει την ουσιαστική αξιολόγηση της ικανότητας των μοντέλων Perceptron και Softmax να διαχωρίζουν μη πλήρως γραμμικά διαχωρίσιμες κλάσεις.



Εικόνα 1-1Ενδεικτικά δείγματα εικόνων από το dataset Fashion-MNIST

2. Προ-επεξεργασία

Κάθε εικόνα του συνόλου δεδομένων αναπαρίσταται ως ένας πίνακας διαστάσεων 28×28, όπου κάθε στοιχείο αντιστοιχεί στην ένταση φωτεινότητας ενός pixel και λαμβάνει τιμές στο διάστημα [0,255].

Για τις ανάγκες της παρούσας εργασίας, εφαρμόστηκε προεπεξεργασία των δεδομένων, η οποία περιλαμβάνει κανονικοποίηση των τιμών των pixels στο διάστημα [0,1], με στόχο τη βελτίωση της σύγκλισης των αλγορίθμων εκπαίδευσης και τη σταθερότητα της διαδικασίας μάθησης.

Επιπλέον, οι εικόνες μετατράπηκαν σε μονοδιάστατα διανύσματα (flattening) διάστασης 784, ώστε να είναι συμβατές με τους γραμμικούς ταξινομητές που χρησιμοποιούνται στην εργασία, όπως ο Perceptron και η Softmax Regression, οι οποίοι δέχονται ως είσοδο διανύσματα χαρακτηριστικών.

Η διαδικασία αυτή επιτρέπει την απλοποίηση της αναπαράστασης των δεδομένων, διατηρώντας παράλληλα την απαραίτητη πληροφορία για την εκπαίδευση των μοντέλων.

3. Θεωρητικό Υπόβαθρο

Η ταξινόμηση αποτελεί ένα από τα βασικότερα προβλήματα της εποπτευόμενης μάθησης, όπου στόχος είναι η εκχώρηση μιας ετικέτας κατηγορίας σε ένα δεδομένο δείγμα εισόδου. Στην παρούσα εργασία, εξετάζονται γραμμικοί ταξινομητές, οι οποίοι αποτελούν απλές αλλά ισχυρές μεθόδους για προβλήματα ταξινόμησης.

3.1 Γραμμικοί Ταξινομητές (Linear Classifiers)

Οι γραμμικοί ταξινομητές βασίζονται στην ιδέα ότι τα δεδομένα μπορούν να διαχωριστούν μέσω ενός γραμμικού ορίου απόφασης. Συγκεκριμένα, για ένα διάνυσμα χαρακτηριστικών x, η απόφαση λαμβάνεται μέσω μιας γραμμικής συνάρτησης:

$$f(x) = w^T x + b$$

όπου w είναι το διάνυσμα βαρών και b η σταθερά μετατόπισης (bias).

Η ταξινόμηση ενός δείγματος προκύπτει από το πρόσημο της συνάρτησης:

- Αν f(x)>0, το δείγμα ανήκει σε μία κλάση
- Αν f(x)<0, ανήκει στην άλλη

Οι γραμμικοί ταξινομητές είναι ιδιαίτερα αποδοτικοί υπολογιστικά, ωστόσο περιορίζονται σε προβλήματα όπου τα δεδομένα είναι γραμμικά διαχωρίσιμα ή περίπου γραμμικά.

3.2 Perceptron

Ο Perceptron αποτελεί έναν από τους πρώτους αλγορίθμους εποπτευόμενης μάθησης και χρησιμοποιείται για την εκμάθηση γραμμικών ορίων απόφασης. Η εκπαίδευση πραγματοποιείται μέσω επαναληπτικής ενημέρωσης των βαρών.

3.2.1 Δυαδικός Perceptron

Για δυαδική ταξινόμηση, ο Perceptron προβλέπει:

$$\hat{y} = \text{sign}(w^T x + b)$$

Όταν ένα δείγμα ταξινομείται λανθασμένα, τα βάρη ενημερώνονται ως εξής:

$$w \leftarrow w + \eta y\, x$$

$$b \leftarrow b + \eta y$$

όπου:

- η: ρυθμός μάθησης
- y: πραγματική ετικέτα

3.2.2 Perceptron One-vs-All (OvA)

Στην περίπτωση πολλαπλών κλάσεων, η προσέγγιση One-vs-All εκπαιδεύει έναν ταξινομητή για κάθε κλάση. Κάθε μοντέλο μαθαίνει να διαχωρίζει μία κλάση από όλες τις υπόλοιπες.

Η τελική πρόβλεψη προκύπτει από:

$$\hat{y} = \arg\max_k (w_k^T x + b_k)$$

3.2.3 Perceptron One-vs-One (OvO)

Στην προσέγγιση One-vs-One, εκπαιδεύεται ένας ταξινομητής για κάθε ζεύγος κλάσεων. Για K κλάσεις, οι ταξινομητές που απαιτούνται είναι:

$$\frac{K(K-1)}{2}$$

Η τελική απόφαση λαμβάνεται μέσω ψηφοφορίας (voting) μεταξύ των επιμέρους ταξινομητών.

3.2.4 Softmax Regression

Η Softmax Regression αποτελεί γενίκευση της λογιστικής παλινδρόμησης για προβλήματα πολλαπλών κλάσεων. Αντί για δυαδική απόφαση, επιστρέφει πιθανότητες για κάθε κλάση.
Η πιθανότητα μια είσοδος x να ανήκει στην κλάση k δίνεται από:

$$P(y = k \mid x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

Η τελική πρόβλεψη προκύπτει από τη μέγιστη πιθανότητα:

$$\hat{y} = \arg\max_k P(y = k \mid x)$$

Η εκπαίδευση του μοντέλου γίνεται μέσω ελαχιστοποίησης της συνάρτησης κόστους cross-entropy, η οποία επιτρέπει την αποτελεσματική προσαρμογή των βαρών ώστε να μεγιστοποιείται η πιθανότητα της σωστής κλάσης.

Για την εκπαίδευση του μοντέλου Softmax Regression χρησιμοποιείται η συνάρτηση κόστους **cross-entropy**, η οποία μετρά την απόκλιση μεταξύ των πραγματικών ετικετών και των προβλεπόμενων πιθανοτήτων.
Για ένα σύνολο δεδομένων με N δείγματα και C κλάσεις, η συνάρτηση κόστους ορίζεται ως:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(p_{i,c})$$

όπου:

- $y_{i,c}$ είναι η πραγματική ετικέτα (one-hot encoding) του δείγματος i,
- $p_{i,c} = P(y = c|x_i)$ είναι η πιθανότητα που προβλέπει το μοντέλο ότι το δείγμα x_i ανήκει στην κλάση c.

Η συνάρτηση cross-entropy τιμωρεί έντονα τις λανθασμένες προβλέψεις, ειδικά όταν το μοντέλο αποδίδει χαμηλή πιθανότητα στη σωστή κλάση, οδηγώντας έτσι σε πιο αποδοτική εκπαίδευση.
Για την αποφυγή υπερπροσαρμογής (**overfitting**), προστίθεται όρος κανονικοποίησης L2 στη συνάρτηση κόστους, ο οποίος περιορίζει το μέγεθος των βαρών του μοντέλου:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(p_{i,c}) + \lambda|W|^2$$

όπου:

- λ είναι η παράμετρος κανονικοποίησης (regularization parameter),

- $||W||^2 = \sum_j W_j^2$ είναι το τετράγωνο του Ευκλείδειου κανόνα των βαρών.

Η κανονικοποίηση L2 επιβάλλει μικρότερες τιμές στα βάρη, αποτρέποντας το μοντέλο από το να προσαρμόζεται υπερβολικά στα δεδομένα εκπαίδευσης και βελτιώνοντας τη γενίκευση σε νέα δεδομένα.

Η ελαχιστοποίηση της συνάρτησης κόστους πραγματοποιείται με τη μέθοδο **gradient descent**, κατά την οποία τα βάρη ενημερώνονται επαναληπτικά σύμφωνα με την κλίση της συνάρτησης κόστους:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W L\left(W^{(t)}\right)$$

όπου:

- η είναι ο ρυθμός μάθησης (learning rate),
- $\nabla_W L(W^{(t)})$ είναι η κλίση της συνάρτησης κόστους ως προς τα βάρη.

Η διαδικασία αυτή οδηγεί στη σταδιακή σύγκλιση του μοντέλου σε ένα σύνολο βαρών που ελαχιστοποιεί το σφάλμα πρόβλεψης.

4. Ποιοτική Ανάλυση Αποτελεσμάτων

Σε αυτό το κεφάλαιο παρουσιάζεται ποιοτική ανάλυση των αποτελεσμάτων των μοντέλων Perceptron (OVA, OVO) και Softmax Regression. Στόχος είναι η κατανόηση της συμπεριφοράς των μοντέλων πέρα από τις αριθμητικές μετρικές αξιολόγησης, μέσω της οπτικής επιθεώρησης σωστών και λανθασμένων ταξινομήσεων.

Η ανάλυση αυτή συμβάλλει στον εντοπισμό προτύπων επιτυχίας και σφαλμάτων, καθώς και στην κατανόηση των περιορισμών των γραμμικών μοντέλων.

4.1 Ποιοτική Ανάλυση Αποτελεσμάτων

Οι παρακάτω απεικονίσεις παρουσιάζουν ενδεικτικά παραδείγματα σωστών και λανθασμένων προβλέψεων για τα μοντέλα **Perceptron (OVA, OVO)** και **Softmax Regression**, τόσο για το ισορροπημένο (balanced) όσο και για το μη ισορροπημένο (imbalanced) σύνολο δεδομένων.

Στην πρώτη κατηγορία απεικονίζονται δείγματα που ταξινομήθηκαν σωστά (Correct Classifications), όπου η προβλεπόμενη κλάση (Predicted) ταυτίζεται με την πραγματική κλάση (True). Παρατηρείται ότι όλα τα μοντέλα αποδίδουν ιδιαίτερα καλά σε κατηγορίες με έντονα και διακριτά μορφολογικά χαρακτηριστικά, όπως:

- **Sneaker**
- **Ankle boot**
- **Bag**
- **Trouser**

Τα αντικείμενα αυτά παρουσιάζουν σαφή γεωμετρική δομή και λιγότερη εσωτερική ποικιλία, γεγονός που διευκολύνει τον γραμμικό διαχωρισμό τους στον χώρο χαρακτηριστικών.

Αντίθετα, στη δεύτερη κατηγορία παρουσιάζονται δείγματα που ταξινομήθηκαν λανθασμένα (Incorrect Classifications), όπου παρατηρείται σημαντική απόκλιση μεταξύ πραγματικής και προβλεπόμενης κλάσης. Τα σφάλματα εντοπίζονται κυρίως σε κατηγορίες με υψηλή οπτική ομοιότητα, όπως:

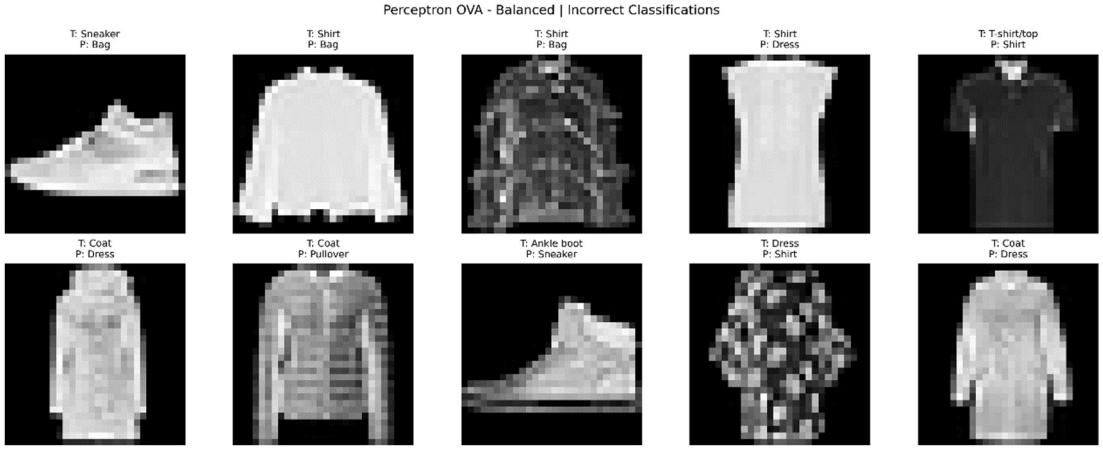
- **T-shirt/top ↔ Shirt**
- **Pullover ↔ Coat**
- **Dress ↔ Shirt**

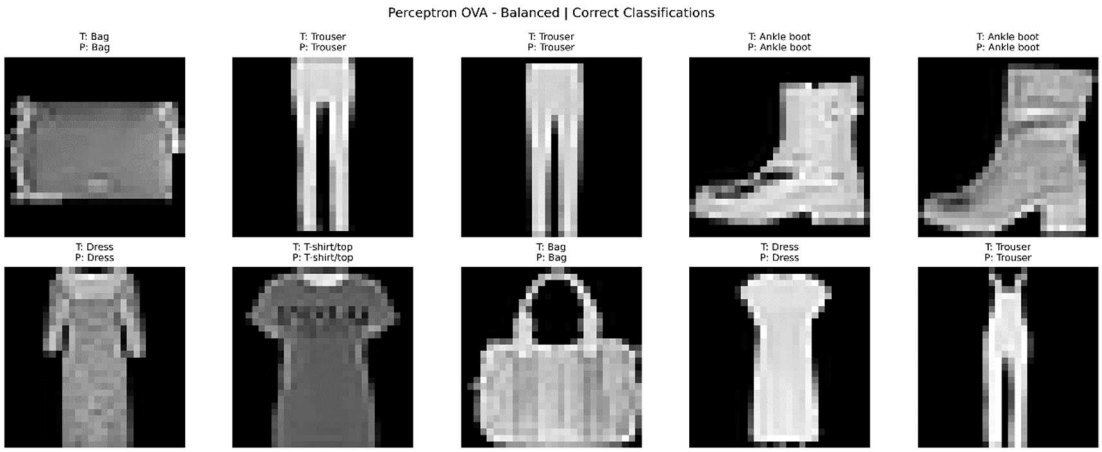
Οι συγκεκριμένες κατηγορίες εμφανίζουν παρόμοια κατανομή pixel και κοινά χαρακτηριστικά (π.χ. σχήμα κορμού, μανίκια) γεγονός που καθιστά δύσκολο τον διαχωρισμό τους μέσω γραμμικών μοντέλων.

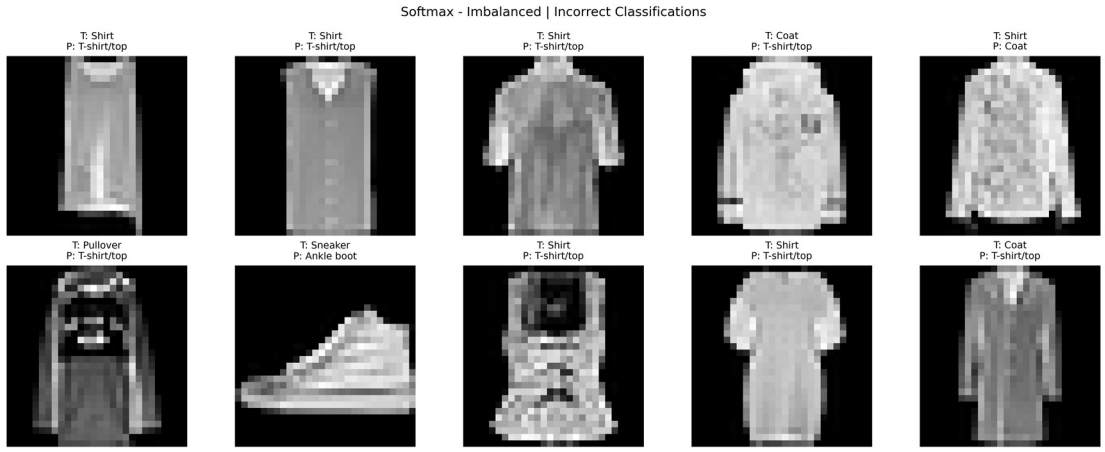
Επιπλέον, παρατηρείται ότι ορισμένα δείγματα είναι εγγενώς αμφίσημα, ακόμη και για ανθρώπινη ερμηνεία, ιδιαίτερα όταν:

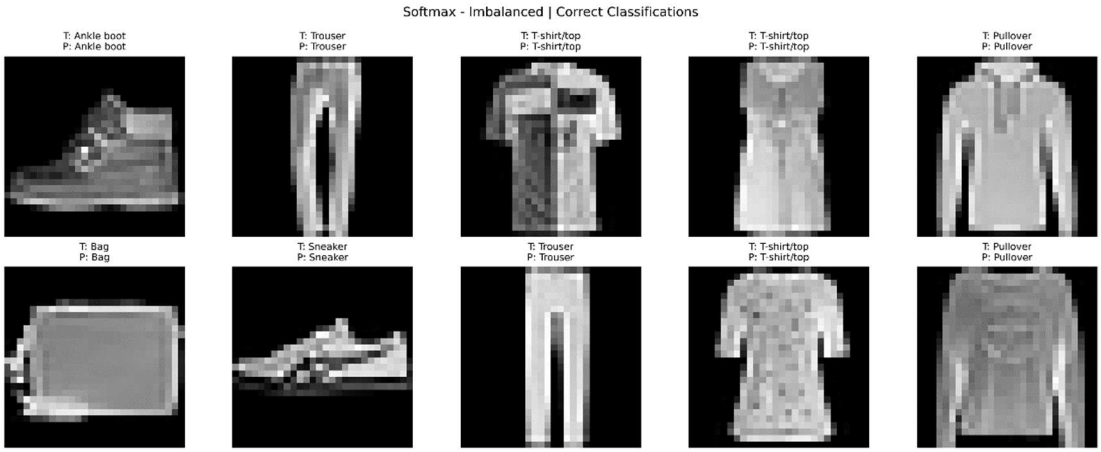
- η εικόνα είναι θορυβώδης
- το αντικείμενο δεν είναι πλήρως ορατό
- υπάρχουν ασυνήθιστες γωνίες ή μορφές.

Η ποιοτική αυτή ανάλυση επιβεβαιώνει ότι τα γραμμικά μοντέλα, παρά την ικανοποιητική συνολική τους απόδοση, παρουσιάζουν περιορισμούς σε προβλήματα όπου οι κλάσεις δεν είναι γραμμικά διαχωρίσιμες.









4.2 Ανάλυση Confusion Matrices

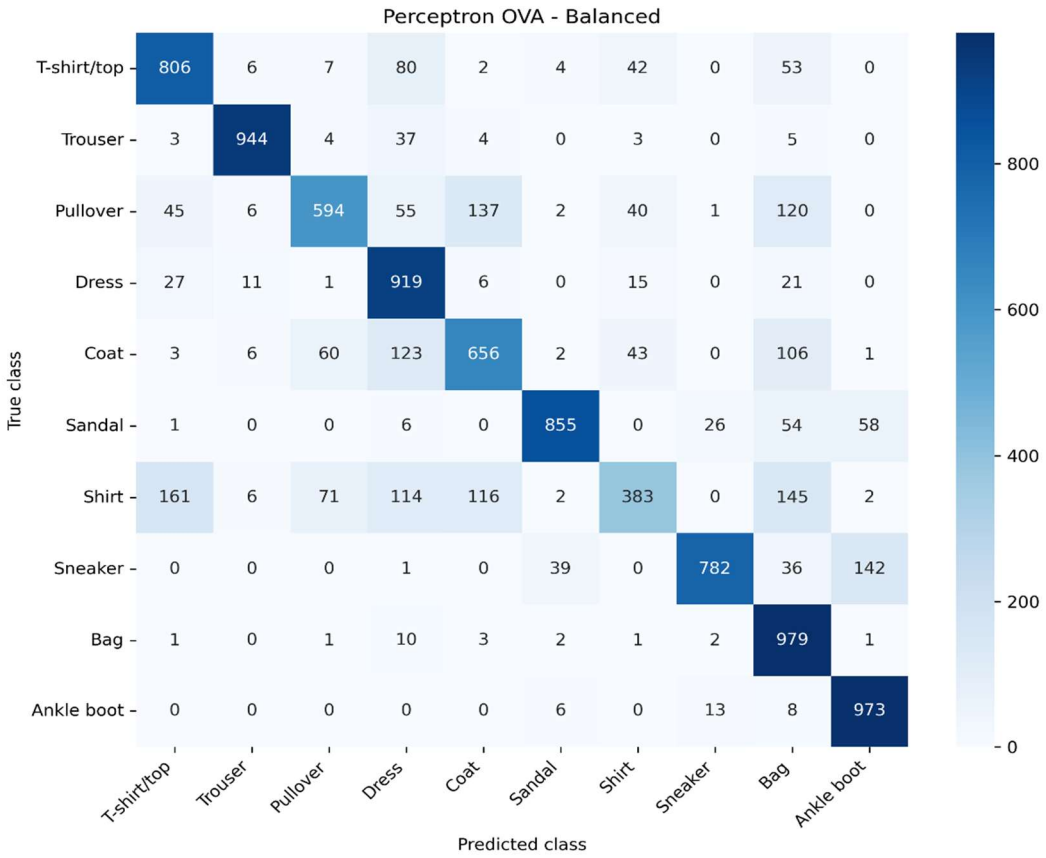
Στα παραπάνω σχήματα παρουσιάζονται οι πίνακες σύγχυσης (confusion matrices) για τα μοντέλα *Perceptron OVA*, *Perceptron OVO* και *Softmax Regression*, σε balanced και imbalanced σενάρια.

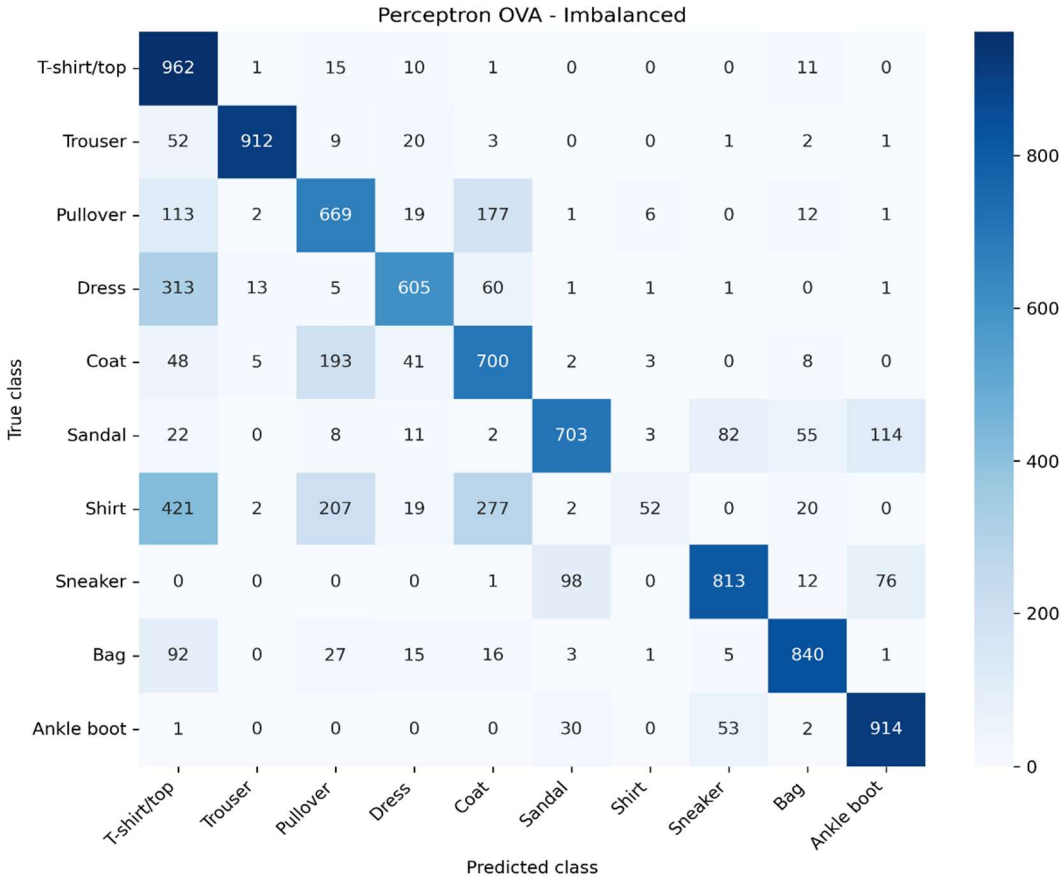
4.2.1 Perceptron OVA (One-vs-All)

Το Perceptron OVA παρουσιάζει ικανοποιητική απόδοση στις περισσότερες κλάσεις, με υψηλές τιμές στη διαγώνιο του πίνακα. Ωστόσο, εμφανίζει σημαντική σύγχυση μεταξύ κατηγοριών με παρόμοια χαρακτηριστικά, ιδιαίτερα στις:

- Shirt
- Pullover
- Coat

Στο imbalanced dataset, παρατηρείται έντονη μεροληψία προς την κυρίαρχη κλάση, γεγονός που οδηγεί σε αυξημένα σφάλματα στις υπόλοιπες κατηγορίες.


Perceptron OVA - Balanced
True class
Predicted class
T-shirt/top 806 6 7 80 2 4 42 0 53 0
Trouser 3 944 4 37 4 0 3 0 5 0
Pullover 45 6 594 55 137 2 40 1 120 0
Dress 27 11 1 919 6 0 15 0 21 0
Coat 3 6 60 123 656 2 43 0 106 1
Sandal 1 0 0 6 0 855 0 26 54 58
Shirt 161 6 71 114 116 2 383 0 145 2
Sneaker 0 0 0 1 0 39 0 782 36 142
Bag 1 0 1 10 3 2 1 2 979 1
Ankle boot 0 0 0 0 0 6 0 13 8 973
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
0 200 400 600 800

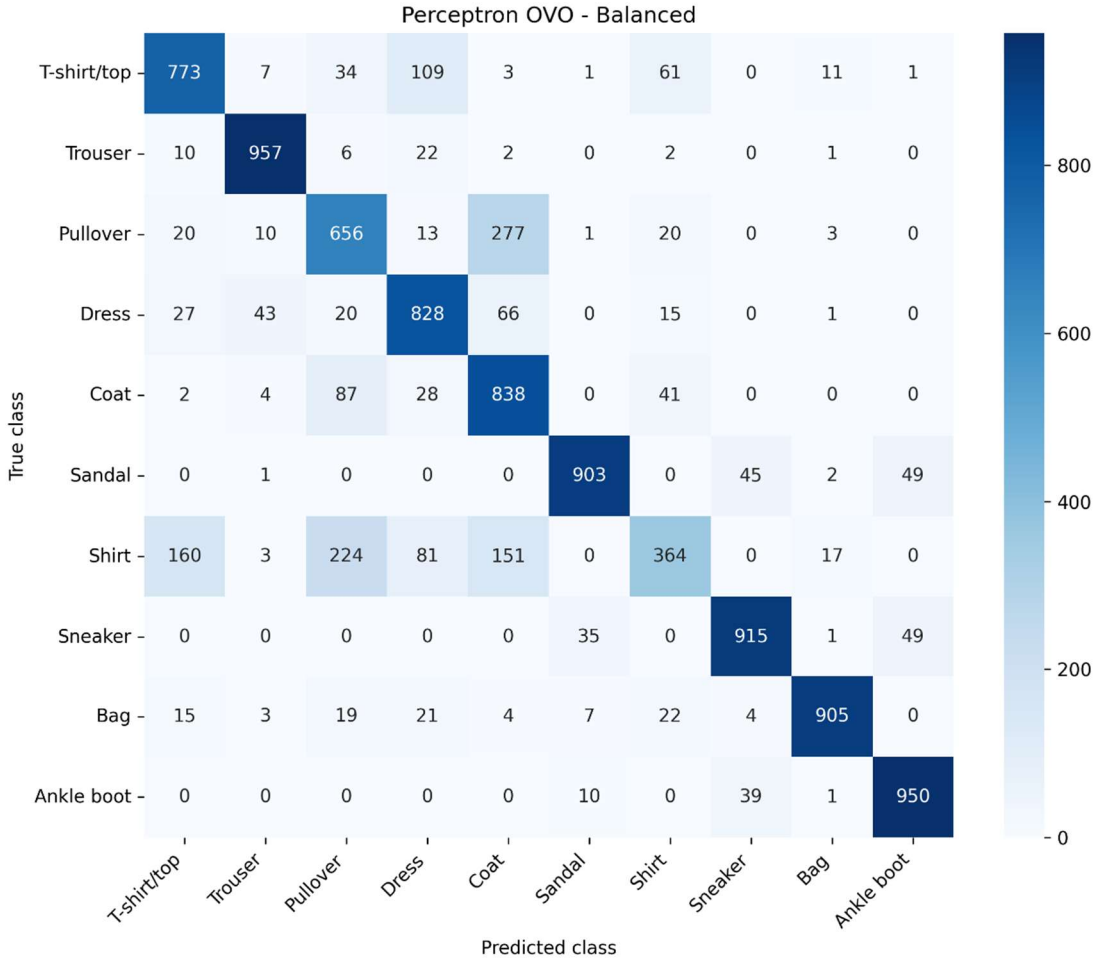

Perceptron OVA - Imbalanced
True class
Predicted class
T-shirt/top 962 1 15 10 1 0 0 0 11 0
Trouser 52 912 9 20 3 0 0 1 2 1
Pullover 113 2 669 19 177 1 6 0 12 1
Dress 313 13 5 605 60 1 1 1 0 1
Coat 48 5 193 41 700 2 3 0 8 0
Sandal 22 0 8 11 2 703 3 82 55 114
Shirt 421 2 207 19 277 2 52 0 20 0
Sneaker 0 0 0 0 1 98 0 813 12 76
Bag 92 0 27 15 16 3 1 5 840 1
Ankle boot 1 0 0 0 0 30 0 53 2 914
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
0 200 400 600 800

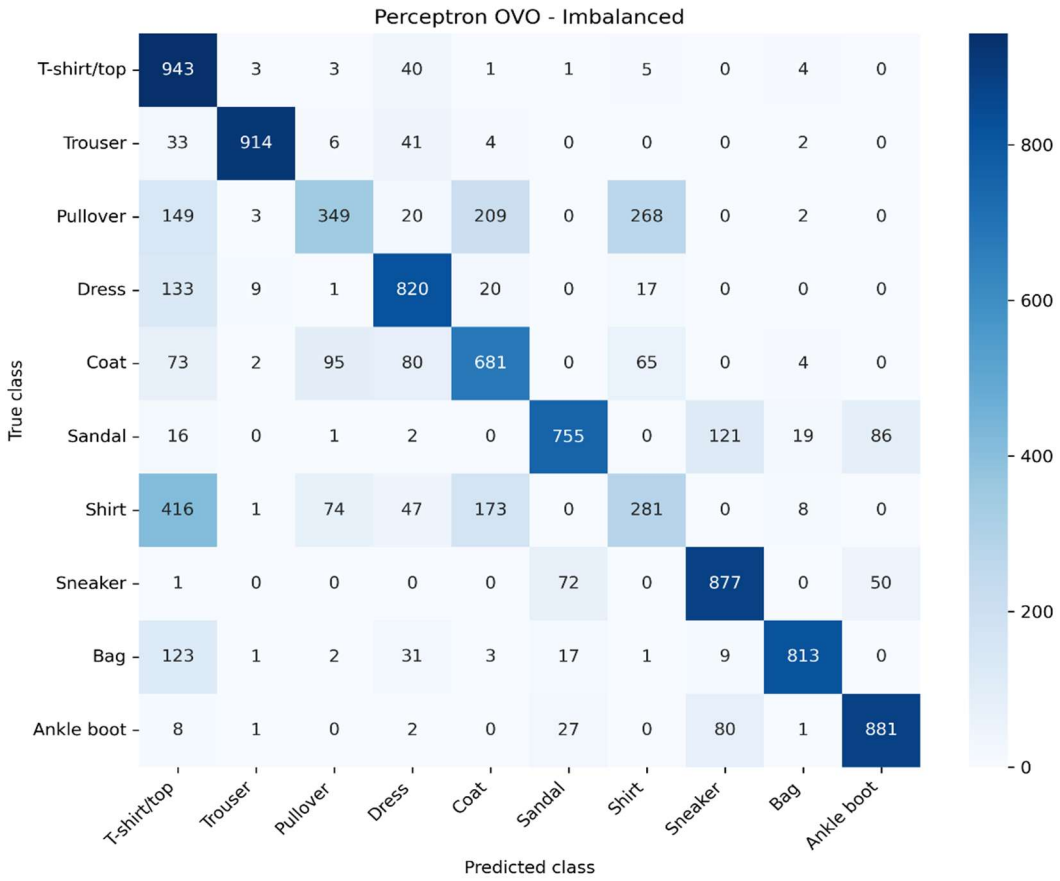
4.2.2 Perceptron OVO (One-vs-One)

Το Perceptron OVO εμφανίζει βελτιωμένη απόδοση σε σχέση με το OVA, καθώς η εκπαίδευση γίνεται σε δυαδικά προβλήματα μεταξύ ζευγών κλάσεων. Αυτό οδηγεί σε:

- καλύτερο διαχωρισμό μεταξύ παρόμοιων κατηγοριών,
- αυξημένη ακρίβεια σε δύσκολες περιπτώσεις.

Παρόλα αυτά, η σύγχυση μεταξύ κατηγοριών όπως *Shirt–T-shirt/top* και *Pullover–Coat* εξακολουθεί να υφίσταται.





4.2.3 Softmax Regression

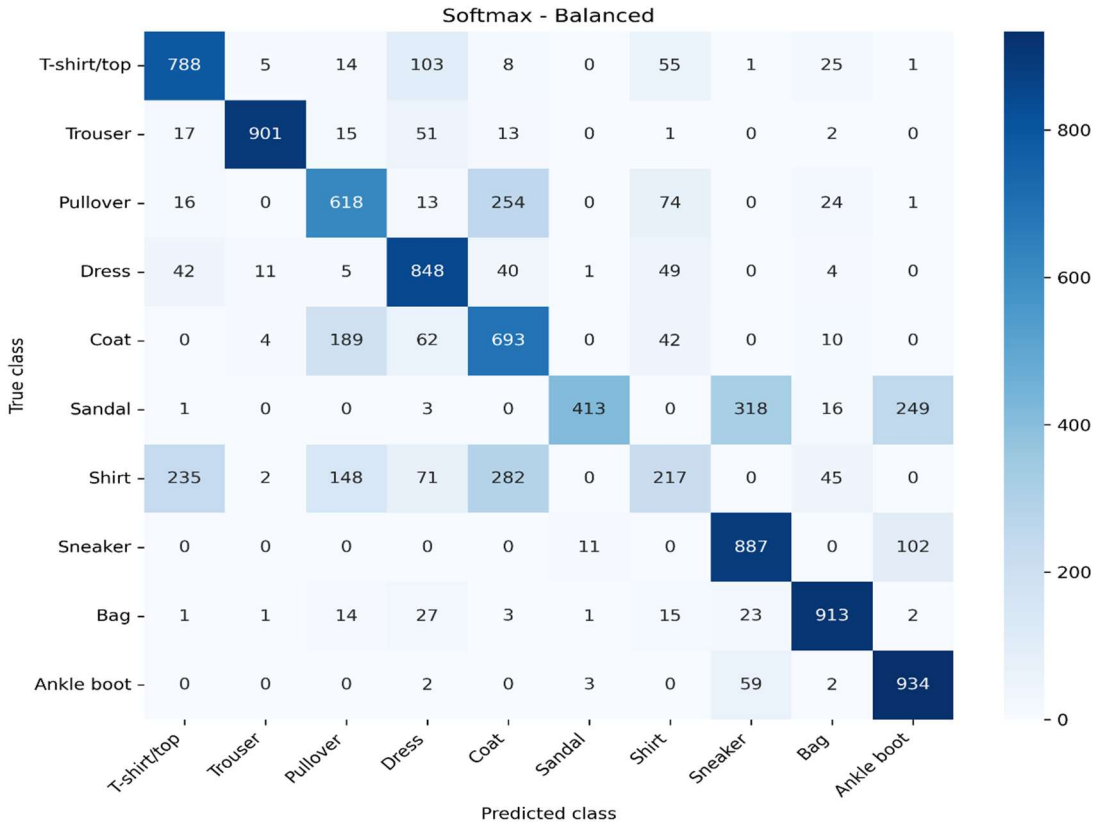
Το Softmax μοντέλο παρουσιάζει συνολικά πιο σταθερή και ισορροπημένη συμπεριφορά, καθώς:

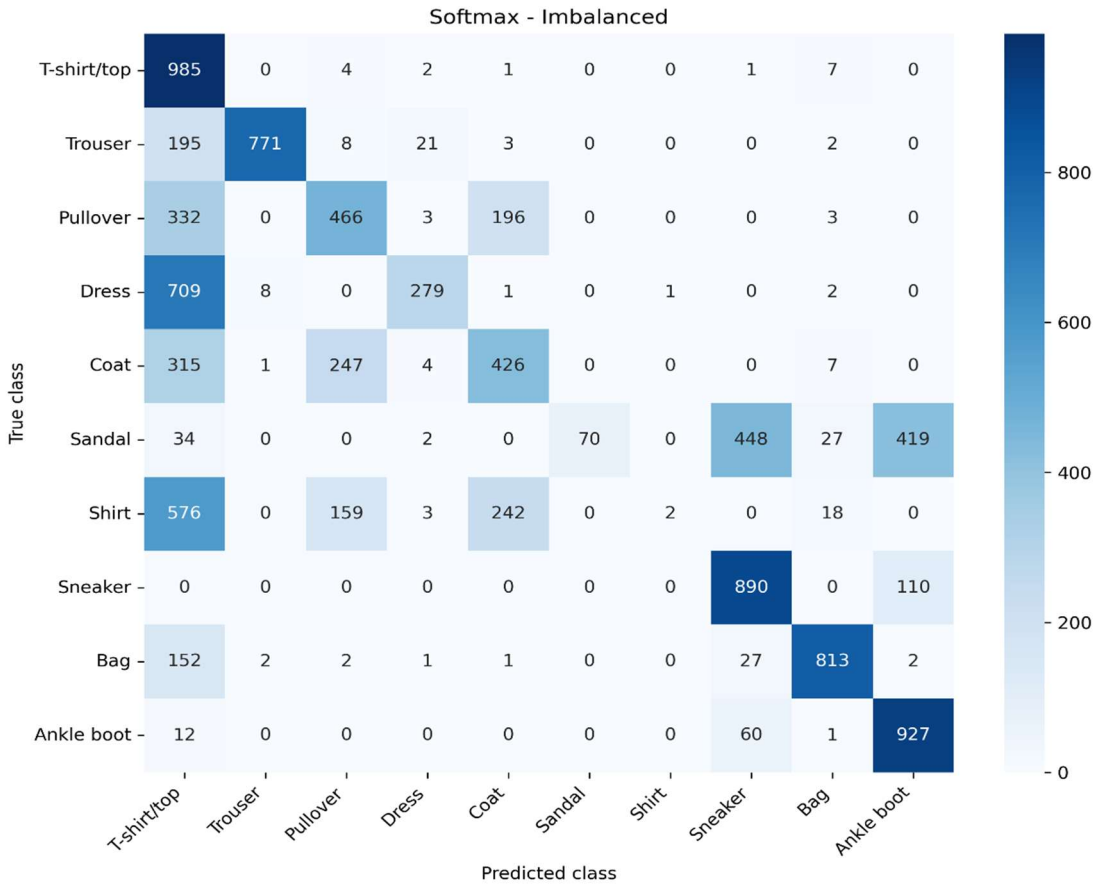
- μαθαίνει ταυτόχρονα όλες τις κλάσεις,
- εκτιμά πιθανότητες αντί για δυαδικές αποφάσεις.

Στο balanced dataset επιτυγχάνει υψηλή ακρίβεια στις περισσότερες κατηγορίες, ενώ στο imbalanced dataset εμφανίζει καλύτερη γενίκευση σε σχέση με τα Perceptron μοντέλα, αν και εξακολουθεί να επηρεάζεται από την ανισορροπία.

Ιδιαίτερα έντονη σύγχυση παρατηρείται στην κατηγορία *Sandal*, η οποία συχνά συγχέεται με:

- Sneaker
- Ankle boot


Softmax - Balanced
True class
Predicted class
T-shirt/top 788 5 14 103 8 0 55 1 25 1
Trouser 17 901 15 51 13 0 1 0 2 0
Pullover 16 0 618 13 254 0 74 0 24 1
Dress 42 11 5 848 40 1 49 0 4 0
Coat 0 4 189 62 693 0 42 0 10 0
Sandal 1 0 0 3 0 413 0 318 16 249
Shirt 235 2 148 71 282 0 217 0 45 0
Sneaker 0 0 0 0 0 11 0 887 0 102
Bag 1 1 14 27 3 1 15 23 913 2
Ankle boot 0 0 0 2 0 3 0 59 2 934
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
0 200 400 600 800


Softmax - Imbalanced
True class
Predicted class
T-shirt/top 985 0 4 2 1 0 0 1 7 0
Trouser 195 771 8 21 3 0 0 0 2 0
Pullover 332 0 466 3 196 0 0 0 3 0
Dress 709 8 0 279 1 0 1 0 2 0
Coat 315 1 247 4 426 0 0 0 7 0
Sandal 34 0 0 2 0 70 0 448 27 419
Shirt 576 0 159 3 242 0 2 0 18 0
Sneaker 0 0 0 0 0 0 0 890 0 110
Bag 152 2 2 1 1 0 0 27 813 2
Ankle boot 12 0 0 0 0 0 0 60 1 927
T-shirt/top Trouser Pullover Dress Coat Sandal Shirt Sneaker Bag Ankle boot
0 200 400 600 800

4.3 Συμπεράσματα από την Ποιοτική Ανάλυση

Συνολικά, η ανάλυση των σωστών και λανθασμένων προβλέψεων δείχνει ότι:

- Τα γραμμικά μοντέλα αποδίδουν εξαιρετικά σε κατηγορίες με σαφή χαρακτηριστικά.
- Δυσκολεύονται σε κατηγορίες με υψηλή οπτική επικάλυψη.
- Το *OVO* υπερέχει του *OVA* σε δύσκολα προβλήματα.
- Το *Softmax* παρουσιάζει πιο ομαλή και γενικεύσιμη συμπεριφορά.
- Η ανισορροπία δεδομένων επηρεάζει σημαντικά την απόδοση όλων των μοντέλων.

Η οπτική ανάλυση συμπληρώνει ουσιαστικά την ποσοτική αξιολόγηση και προσφέρει βαθύτερη κατανόηση της συμπεριφοράς των μοντέλων.

5. Αποτελέσματα

5.1 Εισαγωγή

Σε αυτό το κεφάλαιο παρουσιάζονται τα πειραματικά αποτελέσματα των γραμμικών ταξινομητών που υλοποιήθηκαν, και συγκεκριμένα των:

- Perceptron One-vs-All (OVA)
- Perceptron One-vs-One (OVO)
- Softmax Regression

Η αξιολόγηση πραγματοποιήθηκε στο dataset **Fashion-MNIST**, υπό δύο διαφορετικά σενάρια:

- Ισορροπημένο σύνολο δεδομένων (balanced dataset)
- Μη ισορροπημένο σύνολο δεδομένων (imbalanced dataset)

Για κάθε μοντέλο εξετάστηκε:

- η εξέλιξη της ακρίβειας (accuracy) κατά την εκπαίδευση
- η πορεία του σφάλματος (training error ή loss)
- η τελική απόδοση στο test set

5.2 Αποτελέσματα σε Balanced Dataset

Στο ισορροπημένο σύνολο δεδομένων (balanced dataset), όλα τα μοντέλα παρουσιάζουν σαφή τάση σύγκλισης κατά τη διάρκεια της εκπαίδευσης, γεγονός που υποδηλώνει ότι οι επιλεγμένες υπερπαράμετροι οδηγούν σε σταθερή διαδικασία μάθησης.

Το Perceptron One-vs-All (OVA) εμφανίζει μικρές διακυμάνσεις στην ακρίβεια μεταξύ των εποχών, με τιμές που κυμαίνονται σε σχετικά υψηλά επίπεδα. Οι διακυμάνσεις αυτές αποδίδονται στη φύση του αλγορίθμου perceptron, ο οποίος ενημερώνει τα βάρη μόνο σε περιπτώσεις λανθασμένης ταξινόμησης, οδηγώντας σε μη ομαλή εξέλιξη της συνάρτησης κόστους και της ακρίβειας.

Το Perceptron One-vs-One (OVO) παρουσιάζει πιο σταθερή συμπεριφορά και ελαφρώς καλύτερη σύγκλιση. Η εκπαίδευση σε δυαδικά προβλήματα για κάθε ζεύγος

κλάσεων επιτρέπει πιο ακριβή προσδιορισμό των ορίων απόφασης, ιδιαίτερα σε περιπτώσεις όπου οι κλάσεις παρουσιάζουν οπτική ομοιότητα.

Αντίστοιχα, η Softmax Regression εμφανίζει την πιο ομαλή και σταθερή καμπύλη εκπαίδευσης. Η ακρίβεια αυξάνεται προοδευτικά, ενώ η συνάρτηση κόστους (cross-entropy loss) μειώνεται μονοτονικά, γεγονός που υποδεικνύει αποτελεσματική βελτιστοποίηση μέσω gradient descent.

Τα τελικά αποτελέσματα στο test set για το balanced dataset συνοψίζονται ως εξής:

- Perceptron OVA: 78.13%
- Perceptron OVO: 79.97%
- Softmax Regression: 76.36%

Παρατηρείται ότι το Perceptron OVO επιτυγχάνει την υψηλότερη ακρίβεια, γεγονός που επιβεβαιώνει την υπεροχή της προσέγγισης One-vs-One σε πολυκατηγορικά προβλήματα. Η δυνατότητα εκπαίδευσης εξειδικευμένων δυαδικών ταξινομητών οδηγεί σε καλύτερη διάκριση μεταξύ των κλάσεων.

Το Perceptron OVA ακολουθεί με ελαφρώς χαμηλότερη απόδοση, ενώ η Softmax Regression παρουσιάζει συγκρίσιμη αλλά ελαφρώς μειωμένη ακρίβεια. Παρόλο που η Softmax παρέχει πιο σταθερή και θεωρητικά θεμελιωμένη διαδικασία εκπαίδευσης μέσω πιθανοκρατικής μοντελοποίησης, η γραμμική της φύση περιορίζει την ικανότητά της να προσαρμόζεται σε πιο σύνθετα μοτίβα των δεδομένων εικόνας.

Συνολικά, τα αποτελέσματα δείχνουν ότι, σε ισορροπημένα δεδομένα, οι perceptron-based προσεγγίσεις μπορούν να ανταγωνιστούν — και σε ορισμένες περιπτώσεις να ξεπεράσουν — πιο "θεωρητικά ισχυρά" μοντέλα όπως η Softmax Regression, όταν το πρόβλημα παραμένει γραμμικά προσεγγίσιμο.

5.3 Αποτελέσματα σε Imbalanced Dataset

Στο μη ισορροπημένο σύνολο δεδομένων (imbalanced dataset) παρατηρείται διαφοροποιημένη συμπεριφορά των μοντέλων, καθώς η παρουσία κυρίαρχης κλάσης επηρεάζει σημαντικά τη διαδικασία εκπαίδευσης και την τελική απόδοση.

Το Perceptron One-vs-All (OVA) εμφανίζει ταχεία αύξηση της ακρίβειας στις πρώτες εποχές, φτάνοντας σε υψηλές τιμές στο training set. Το φαινόμενο αυτό οφείλεται στη μεροληψία προς την κυρίαρχη κλάση, καθώς το μοντέλο τείνει να προβλέπει συχνότερα την πλειοψηφική κατηγορία. Ωστόσο, η συμπεριφορά αυτή δε μεταφράζεται σε αντίστοιχα υψηλή απόδοση στο test set, υποδηλώνοντας περιορισμένη ικανότητα γενίκευσης.

Το Perceptron One-vs-One (OVO) παρουσιάζει πιο σταθερή και ισορροπημένη βελτίωση κατά την εκπαίδευση. Η εκπαίδευση σε δυαδικά προβλήματα ανά ζεύγος κλάσεων μειώνει εν μέρει την επίδραση της ανισορροπίας, οδηγώντας σε καλύτερη συνολική απόδοση σε σχέση με το OVA.

Αντίθετα με την αρχική εκτίμηση, η Softmax Regression εμφανίζει ανταγωνιστική — και σε αυτή την περίπτωση ανώτερη — απόδοση σε σχέση με τα perceptron μοντέλα. Η χρήση class weighting κατά την εκπαίδευση επιτρέπει στο μοντέλο να αντισταθμίσει την ανισορροπία των δεδομένων, δίνοντας μεγαλύτερη σημασία στις υποεκπροσωπούμενες κλάσεις.

Τα αποτελέσματα στο test set για το imbalanced dataset συνοψίζονται ως εξής:

- Perceptron OVA: 67.32%
- Perceptron OVO: 71.31%
- Softmax Regression: 75.17%

Παρατηρείται ότι η Softmax Regression επιτυγχάνει την υψηλότερη ακρίβεια, γεγονός που υπογραμμίζει τη σημασία της ενσωμάτωσης μηχανισμών αντιμετώπισης

της ανισορροπίας, όπως τα class weights. Αντίθετα, τα perceptron μοντέλα, τα οποία δεν διαθέτουν εγγενή μηχανισμό διαχείρισης ανισορροπημένων δεδομένων, παρουσιάζουν μεγαλύτερη ευαισθησία στη μεταβολή της κατανομής των κλάσεων.

Η συνολική μείωση της απόδοσης σε σχέση με το balanced dataset επιβεβαιώνει ότι η ανισορροπία των δεδομένων αποτελεί κρίσιμο παράγοντα που επηρεάζει την απόδοση των μοντέλων. Επιπλέον, αναδεικνύεται ότι η συνολική ακρίβεια (accuracy) μπορεί να είναι παραπλανητική σε τέτοιες περιπτώσεις, καθώς ένα μοντέλο μπορεί να επιτυγχάνει υψηλή ακρίβεια προβλέποντας κυρίως την κυρίαρχη κλάση.

Συνολικά, τα αποτελέσματα δείχνουν ότι η κατάλληλη προσαρμογή του μοντέλου στην κατανομή των δεδομένων είναι καθοριστικής σημασίας για την επίτευξη καλής απόδοσης, ιδιαίτερα σε ρεαλιστικά σενάρια όπου η ανισορροπία είναι συχνό φαινόμενο.

5.4 Συγκριτική Ανάλυση

Συνολικά, παρατηρείται ότι:

- Το Perceptron One-vs-One (OVO) επιτυγχάνει την καλύτερη απόδοση στο balanced dataset, επιβεβαιώνοντας την αποτελεσματικότητα της προσέγγισης μέσω δυαδικών ταξινομητών.
- Το Perceptron One-vs-All (OVA) παρουσιάζει παρόμοια αλλά ελαφρώς χαμηλότερη επίδοση, λόγω της πιο γενικευμένης προσέγγισης διαχωρισμού των κλάσεων.
- Η Softmax Regression εμφανίζει συγκρίσιμη απόδοση στο balanced dataset, αλλά υπερέχει στο imbalanced dataset, χάρη στη χρήση μηχανισμών όπως τα class weights.

Η ανισορροπία των δεδομένων επηρεάζει αρνητικά όλα τα μοντέλα, οδηγώντας σε μείωση της συνολικής απόδοσης και εμφάνιση μεροληψίας προς την κυρίαρχη κλάση. Ωστόσο, η επίδραση αυτή δεν είναι ίδια για όλα τα μοντέλα. Τα perceptron μοντέλα, τα οποία δεν ενσωματώνουν εγγενώς μηχανισμούς αντιμετώπισης της ανισορροπίας, παρουσιάζουν μεγαλύτερη ευαισθησία στη μεταβολή της κατανομής των δεδομένων.

Αντίθετα, η Softmax Regression, μέσω της χρήσης class weighting, καταφέρνει να διαχειριστεί πιο αποτελεσματικά την ανισορροπία, διατηρώντας υψηλότερη απόδοση στο imbalanced dataset. Το εύρημα αυτό υπογραμμίζει τη σημασία της ενσωμάτωσης κατάλληλων τεχνικών κατά την εκπαίδευση των μοντέλων.

Τα πειραματικά αποτελέσματα δείχνουν ότι οι γραμμικοί ταξινομητές μπορούν να επιτύχουν ικανοποιητική απόδοση στο Fashion-MNIST, ιδιαίτερα σε ισορροπημένα δεδομένα. Ωστόσο, η απόδοσή τους περιορίζεται σε περιπτώσεις όπου οι κλάσεις δεν είναι εύκολα γραμμικά διαχωρίσιμες ή όταν υπάρχει σημαντική ανισορροπία.

Η ανάλυση αυτή επιβεβαιώνει την ανάγκη για πιο σύνθετα μοντέλα, όπως τα νευρωνικά δίκτυα, τα οποία μπορούν να μοντελοποιήσουν μη γραμμικές σχέσεις και να αξιοποιήσουν πιο αποτελεσματικά τη δομή των δεδομένων εικόνας, οδηγώντας σε ανώτερη απόδοση σε προβλήματα υψηλής πολυπλοκότητας.

6. Συμπεράσματα

Η παρούσα εργασία είχε ως βασικό στόχο τη μελέτη και συγκριτική αξιολόγηση γραμμικών ταξινομητών σε πρόβλημα πολυκατηγορικής ταξινόμησης εικόνων, χρησιμοποιώντας το σύνολο δεδομένων Fashion-MNIST. Συγκεκριμένα, υλοποιήθηκαν από το μηδέν οι αλγόριθμοι Perceptron (στις εκδοχές One-vs-All και One-vs-One) και Softmax Regression, και εξετάστηκε η συμπεριφορά τους υπό διαφορετικές συνθήκες εκπαίδευσης.

Τα πειραματικά αποτελέσματα ανέδειξαν ότι οι γραμμικοί ταξινομητές μπορούν να επιτύχουν ικανοποιητική απόδοση ακόμη και σε προβλήματα εικόνας. Στο ισορροπημένο σύνολο δεδομένων (balanced dataset), η καλύτερη επίδοση επιτεύχθηκε από τον Perceptron One-vs-One με ακρίβεια περίπου 79.97%, ακολουθούμενο από τον Perceptron One-vs-All με 78.13%, ενώ η Softmax Regression παρουσίασε ελαφρώς χαμηλότερη απόδοση της τάξης του 76.36%. Τα αποτελέσματα αυτά καταδεικνύουν ότι, παρά την απλότητά τους, τα γραμμικά μοντέλα είναι ικανά να εκμεταλλευτούν τα διακριτά μορφολογικά χαρακτηριστικά ορισμένων κατηγοριών του dataset.

Ωστόσο, αναδείχθηκε σαφώς ένας θεμελιώδης περιορισμός των γραμμικών μοντέλων: η αδυναμία αποτελεσματικής μοντελοποίησης μη γραμμικά διαχωρίσιμων δεδομένων. Σε κατηγορίες με υψηλή οπτική ομοιότητα, όπως T-shirt/top και Shirt ή Pullover και Coat, παρατηρήθηκαν αυξημένα ποσοστά σφαλμάτων. Το γεγονός αυτό επιβεβαιώνει ότι τα γραμμικά όρια απόφασης δεν επαρκούν για τη σύλληψη πολύπλοκων δομών στον χώρο χαρακτηριστικών, ιδιαίτερα όταν υπάρχει επικάλυψη μεταξύ των κλάσεων.

Η σύγκριση μεταξύ των διαφορετικών προσεγγίσεων του Perceptron έδειξε ότι η στρατηγική One-vs-One υπερτερεί συστηματικά της One-vs-All, τόσο στο balanced όσο και στο imbalanced σενάριο. Συγκεκριμένα, στο μη ισορροπημένο σύνολο δεδομένων (imbalanced dataset), ο Perceptron OVO πέτυχε ακρίβεια 71.31% έναντι 67.32% του OVA . Η υπεροχή αυτή αποδίδεται στο γεγονός ότι η OVO προσέγγιση εκπαιδεύεται σε δυαδικά προβλήματα, επιτρέποντας πιο λεπτομερή διαχωρισμό μεταξύ ζευγών κλάσεων.

Αντίθετα, η Softmax Regression παρουσίασε πιο σταθερή και ομαλή συμπεριφορά κατά την εκπαίδευση, ιδιαίτερα λόγω της πιθανοκρατικής φύσης της και της χρήσης της συνάρτησης softmax. Ενδιαφέρον εύρημα αποτελεί το γεγονός ότι στο imbalanced dataset η απόδοσή της (75.17%) ξεπέρασε αυτή των perceptron μοντέλων, γεγονός που υποδηλώνει ότι η ενσωμάτωση class weighting συμβάλλει ουσιαστικά στη διαχείριση της ανισορροπίας.

Η επίδραση της ανισορροπίας των δεδομένων αποδείχθηκε ιδιαίτερα σημαντική. Παρατηρήθηκε γενικευμένη μείωση της απόδοσης των μοντέλων και έντονη μεροληψία προς την κυρίαρχη κλάση. Το φαινόμενο αυτό αναδεικνύει ότι οι γραμμικοί ταξινομητές είναι ευαίσθητοι στην κατανομή των δεδομένων και ότι η συνολική ακρίβεια δεν αποτελεί πάντα αξιόπιστο δείκτη αξιολόγησης σε τέτοιες περιπτώσεις.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι, παρά τα πλεονεκτήματά τους — όπως η απλότητα, η ταχύτητα και το χαμηλό υπολογιστικό κόστος— οι γραμμικοί ταξινομητές παρουσιάζουν σημαντικούς περιορισμούς σε προβλήματα εικόνας υψηλής πολυπλοκότητας. Η γραμμική τους φύση περιορίζει την ικανότητά τους να συλλάβουν σύνθετες μη γραμμικές σχέσεις μεταξύ των χαρακτηριστικών.

Για τον λόγο αυτό, καθίσταται εμφανής η ανάγκη χρήσης πιο σύνθετων μοντέλων, όπως τα νευρωνικά δίκτυα και ιδιαίτερα τα Συνελικτικά Νευρωνικά Δίκτυα (Convolutional Neural Networks – CNNs), τα οποία είναι ικανά να εξάγουν ιεραρχικά

χαρακτηριστικά και να μοντελοποιούν μη γραμμικές σχέσεις με σημαντικά καλύτερη απόδοση.

Τέλος, ως μελλοντική εργασία προτείνεται:

- η χρήση μη γραμμικών μοντέλων,
- η εφαρμογή τεχνικών αντιμετώπισης ανισορροπίας (π.χ. class weighting, oversampling),
- η διερεύνηση πιο σύνθετων συνόλων δεδομένων.

Η περαιτέρω αυτή διερεύνηση αναμένεται να οδηγήσει σε βαθύτερη κατανόηση της συμπεριφοράς των αλγορίθμων και σε πιο αποδοτικές λύσεις για προβλήματα αναγνώρισης προτύπων.

7. Ανάλυση Κώδικα

```
# ==========================================================
# LINEAR CLASSIFIERS PROJECT - FROM SCRATCH
# Improved version to better cover the full assignment:
# - Perceptron One-vs-All (OVA)
# - Perceptron One-vs-One (OVO)
# - Softmax Regression

# - Author: Grigoris Liatsos
# ==========================================================

import os
import csv
import random
from collections import Counter

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from keras.datasets import fashion_mnist
```

Αρχικά πραγματοποιείται η εισαγωγή των απαραίτητων βιβλιοθηκών που χρησιμοποιούνται για την υλοποίηση των γραμμικών ταξινομητών Perceptron (στις εκδοχές One-vs-All και One-vs-One) και Softmax Regression.

Η βιβλιοθήκη NumPy χρησιμοποιείται για την εκτέλεση αριθμητικών πράξεων και τη διαχείριση πινάκων και διανυσμάτων, που αποτελούν βασικά στοιχεία στην υλοποίηση των μοντέλων. Οι βιβλιοθήκες Matplotlib και Seaborn αξιοποιούνται για την απεικόνιση των αποτελεσμάτων, όπως γραφήματα ακρίβειας και πίνακες σύγχυσης (confusion matrices).

Επιπλέον, οι βιβλιοθήκες os και csv χρησιμοποιούνται για τη διαχείριση αρχείων και την αποθήκευση των πειραματικών αποτελεσμάτων, ενώ η βιβλιοθήκη random αξιοποιείται για την τυχαιοποίηση των δεδομένων. Η δομή Counter χρησιμοποιείται για την καταμέτρηση της κατανομής των κλάσεων, ιδιαίτερα κατά τη δημιουργία ισορροπημένων και μη ισορροπημένων συνόλων δεδομένων.

Τέλος, το σύνολο δεδομένων Fashion-MNIST φορτώνεται μέσω της βιβλιοθήκης Keras και χρησιμοποιείται ως βάση για την εκπαίδευση και αξιολόγηση των μοντέλων.

```
# ============================================================
# REPRODUCIBILITY
# ============================================================
SEED = 42
random.seed(SEED)
np.random.seed(SEED)


# ============================================================
# CONFIG
# ============================================================
RESULTS_DIR = "results_project2_full"
os.makedirs(RESULTS_DIR, exist_ok=True)

NUM_CLASSES = 10
BALANCED_SAMPLES_PER_CLASS = 1000
IMBALANCED_MAIN_CLASS = 0
IMBALANCED_MAIN_SAMPLES = 1000
IMBALANCED_OTHER_SAMPLES = 50

VALIDATION_RATIO = 0.2

# Hyperparameter grids
OVA_LRS = [0.001, 0.01, 0.1]
OVA_EPOCHS_LIST = [5, 10, 20]

OVO_LRS = [0.001, 0.01, 0.1]
OVO_EPOCHS_LIST = [3, 5, 10]

SOFTMAX_LRS = [0.0005, 0.001, 0.003]
SOFTMAX_EPOCHS_LIST = [100, 200, 300]

SHOW_PLOTS = False
SAVE_PLOTS = True


# ============================================================
# CLASS NAMES
# ============================================================
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]
```

Στη συνέχεια ορίζονται βασικές παράμετροι που σχετίζονται με την αναπαραγωγιμότητα των πειραμάτων και τη ρύθμιση της εκπαίδευσης των μοντέλων. Αρχικά, καθορίζεται μια σταθερή τιμή (seed), η οποία χρησιμοποιείται από τις βιβλιοθήκες random και NumPy, ώστε τα αποτελέσματα των πειραμάτων να είναι επαναλήψιμα. Με τον τρόπο αυτό διασφαλίζεται ότι κάθε εκτέλεση του κώδικα οδηγεί στα ίδια αποτελέσματα, διευκολύνοντας τη σύγκριση και την αξιολόγηση των μοντέλων.

Στη συνέχεια ορίζονται παράμετροι διαμόρφωσης (configuration) του πειραματικού πλαισίου. Δημιουργείται ένας φάκελος αποθήκευσης αποτελεσμάτων, στον οποίο αποθηκεύονται τα αρχεία εξόδου, όπως γραφήματα και μετρικές απόδοσης. Επιπλέον, καθορίζεται ο αριθμός των κλάσεων του προβλήματος, καθώς και το πλήθος των δειγμάτων που χρησιμοποιούνται για τη δημιουργία ισορροπημένων και μη ισορροπημένων συνόλων δεδομένων. Στην περίπτωση του imbalanced dataset,

ορίζεται μία κυρίαρχη κλάση με μεγαλύτερο αριθμό δειγμάτων, ενώ οι υπόλοιπες κλάσεις περιλαμβάνουν σημαντικά λιγότερα παραδείγματα.

Ορίζεται επίσης το ποσοστό των δεδομένων που χρησιμοποιείται ως σύνολο επικύρωσης (validation set), το οποίο αξιοποιείται για την αξιολόγηση της απόδοσης των μοντέλων κατά την εκπαίδευση.

Επιπλέον, καθορίζονται πλέγματα υπερπαραμέτρων (hyperparameter grids) για κάθε μοντέλο. Συγκεκριμένα, ορίζονται διαφορετικές τιμές για τον ρυθμό μάθησης (learning rate) και τον αριθμό εποχών (epochs), οι οποίες δοκιμάζονται πειραματικά, με στόχο την εύρεση των βέλτιστων ρυθμίσεων για κάθε αλγόριθμο.

Τέλος, ορίζονται επιλογές που αφορούν την απεικόνιση των αποτελεσμάτων, όπως η εμφάνιση ή αποθήκευση γραφημάτων, καθώς και μία λίστα με τις ονομασίες των κλάσεων του συνόλου δεδομένων Fashion-MNIST, η οποία χρησιμοποιείται για την ερμηνεία και παρουσίαση των αποτελεσμάτων.

```
# ============================================================
# LOAD DATASET
# ============================================================
(X_train_full_raw, y_train_full), (X_test_raw, y_test) =
fashion_mnist.load_data()

# Keep original images for visualization
X_test_images = X_test_raw.copy()

# Flatten + normalize
X_train_full = X_train_full_raw.reshape(X_train_full_raw.shape[0], -
1).astype(np.float32) / 255.0
X_test = X_test_raw.reshape(X_test_raw.shape[0], -
1).astype(np.float32) / 255.0

# ============================================================
# STANDARDIZATION
# ============================================================
mean = np.mean(X_train_full, axis=0)
std = np.std(X_train_full, axis=0) + 1e-8

X_train_full = (X_train_full - mean) / std
X_test = (X_test - mean) / std

# ============================================================
# ADD BIAS FEATURE
# ============================================================
def add_bias_feature(X):
    return np.hstack([X, np.ones((X.shape[0], 1), dtype=X.dtype)])

X_train_full = add_bias_feature(X_train_full)
X_test = add_bias_feature(X_test)
```

Στο επόμενο στάδιο πραγματοποιείται η φόρτωση και η προεπεξεργασία του συνόλου δεδομένων. Το dataset Fashion-MNIST φορτώνεται μέσω της βιβλιοθήκης Keras και διαχωρίζεται σε σύνολο εκπαίδευσης (training set) και σύνολο ελέγχου (test set). Παράλληλα, διατηρείται ένα αντίγραφο των αρχικών εικόνων του test set, το οποίο χρησιμοποιείται αποκλειστικά για σκοπούς οπτικοποίησης των αποτελεσμάτων, όπως η απεικόνιση σωστών και λανθασμένων προβλέψεων.

Στη συνέχεια, εφαρμόζεται προεπεξεργασία των δεδομένων, η οποία περιλαμβάνει τη μετατροπή των εικόνων από δισδιάστατους πίνακες διαστάσεων 28×28 σε

μονοδιάστατα διανύσματα χαρακτηριστικών (flattening). Η διαδικασία αυτή είναι απαραίτητη, καθώς τα γραμμικά μοντέλα δέχονται ως είσοδο διανύσματα και όχι εικόνες. Επιπλέον, πραγματοποιείται κανονικοποίηση των τιμών των pixels, μετατρέποντάς τες από το αρχικό εύρος [0,255] στο διάστημα [0,1], γεγονός που συμβάλλει στη σταθερότητα και την ταχύτερη σύγκλιση της διαδικασίας εκπαίδευσης.

Ακολούθως, εφαρμόζεται τυποποίηση (standardization) των χαρακτηριστικών. Συγκεκριμένα, υπολογίζεται η μέση τιμή και η τυπική απόκλιση για κάθε χαρακτηριστικό με βάση το σύνολο εκπαίδευσης και στη συνέχεια κάθε τιμή μετασχηματίζεται ώστε να έχει μηδενική μέση τιμή και μοναδιαία διασπορά. Η ίδια μετατροπή εφαρμόζεται και στο σύνολο ελέγχου, χρησιμοποιώντας όμως τις παραμέτρους (mean και standard deviation) που υπολογίστηκαν από το training set, ώστε να αποφευχθεί διαρροή πληροφορίας (data leakage).

Τέλος, προστίθεται ένα επιπλέον χαρακτηριστικό στα δεδομένα, το οποίο αντιστοιχεί στον όρο πόλωσης (bias). Η προσθήκη αυτή υλοποιείται μέσω της συνάρτησης add_bias_feature, η οποία επεκτείνει κάθε διάνυσμα εισόδου με μία επιπλέον τιμή ίση με 1. Με τον τρόπο αυτό, ο όρος bias ενσωματώνεται απευθείας στο διάνυσμα χαρακτηριστικών, επιτρέποντας την απλούστερη υλοποίηση των μοντέλων, καθώς δεν απαιτείται ξεχωριστός χειρισμός του κατά τους υπολογισμούς.

Η παραπάνω διαδικασία αποτελεί κρίσιμο στάδιο προετοιμασίας των δεδομένων, διασφαλίζοντας ότι αυτά είναι κατάλληλα μορφοποιημένα και κανονικοποιημένα για την αποτελεσματική εκπαίδευση των γραμμικών ταξινομητών.

```
# ============================================================
# DATASET CREATION
# ============================================================
def create_balanced_dataset(X, y, samples_per_class=1000):
    X_bal, y_bal = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
        chosen = np.random.choice(idx, samples_per_class,
replace=False)
        X_bal.append(X[chosen])
        y_bal.append(y[chosen])

    X_bal = np.vstack(X_bal)
    y_bal = np.hstack(y_bal)

    perm = np.random.permutation(len(y_bal))
    return X_bal[perm], y_bal[perm]


def create_imbalanced_dataset(X, y, main_class=0, main_samples=1000,
other_samples=50):
    X_imb, y_imb = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
        if cls == main_class:
            chosen = np.random.choice(idx, main_samples,
replace=False)
        else:
            chosen = np.random.choice(idx, other_samples,
replace=False)

        X_imb.append(X[chosen])
```

```
        y_imb.append(y[chosen])

    X_imb = np.vstack(X_imb)
    y_imb = np.hstack(y_imb)

    perm = np.random.permutation(len(y_imb))
    return X_imb[perm], y_imb[perm]


def stratified_train_val_split(X, y, val_ratio=0.2):
    X_train_parts, y_train_parts = [], []
    X_val_parts, y_val_parts = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
        idx = np.random.permutation(idx)

        n_val = max(1, int(len(idx) * val_ratio))
        val_idx = idx[:n_val]
        train_idx = idx[n_val:]

        X_val_parts.append(X[val_idx])
        y_val_parts.append(y[val_idx])

        X_train_parts.append(X[train_idx])
        y_train_parts.append(y[train_idx])

    X_train_split = np.vstack(X_train_parts)
    y_train_split = np.hstack(y_train_parts)
    X_val_split = np.vstack(X_val_parts)
    y_val_split = np.hstack(y_val_parts)

    train_perm = np.random.permutation(len(y_train_split))
    val_perm = np.random.permutation(len(y_val_split))

    return (
        X_train_split[train_perm], y_train_split[train_perm],
        X_val_split[val_perm], y_val_split[val_perm]
    )
```

Στο επόμενο στάδιο υλοποιείται η δημιουργία των συνόλων δεδομένων που χρησιμοποιούνται στα πειράματα, καθώς και ο διαχωρισμός τους σε σύνολα εκπαίδευσης και επικύρωσης.

Αρχικά, υλοποιείται η συνάρτηση create_balanced_dataset, η οποία χρησιμοποιείται για τη δημιουργία ενός ισορροπημένου συνόλου δεδομένων (balanced dataset). Στη συνάρτηση αυτή επιλέγεται ο ίδιος αριθμός δειγμάτων από κάθε κλάση, εξασφαλίζοντας ότι όλες οι κατηγορίες εκπροσωπούνται ισότιμα. Η επιλογή των δειγμάτων γίνεται τυχαία, ενώ στο τέλος εφαρμόζεται ανακάτεμα (shuffle) των δεδομένων, ώστε να μην υπάρχει κάποια σειρά που να σχετίζεται με τις κλάσεις.

Στη συνέχεια, υλοποιείται η συνάρτηση create_imbalanced_dataset, η οποία δημιουργεί ένα μη ισορροπημένο σύνολο δεδομένων (imbalanced dataset). Σε αυτή την περίπτωση, ορίζεται μία κυρίαρχη κλάση, από την οποία επιλέγεται μεγαλύτερος αριθμός δειγμάτων, ενώ από τις υπόλοιπες κλάσεις επιλέγεται σημαντικά μικρότερο πλήθος. Με τον τρόπο αυτό προσομοιώνεται ένα ρεαλιστικό σενάριο, όπου οι κλάσεις δεν κατανέμονται ομοιόμορφα, επιτρέποντας τη μελέτη της επίδρασης της ανισορροπίας στην απόδοση των μοντέλων.

Τέλος, υλοποιείται η συνάρτηση stratified_train_val_split, η οποία χρησιμοποιείται για τον διαχωρισμό των δεδομένων σε σύνολο εκπαίδευσης (training set) και σύνολο επικύρωσης (validation set). Ο διαχωρισμός πραγματοποιείται με στρωματοποιημένο τρόπο (stratified), δηλαδή διατηρώντας την ίδια αναλογία κλάσεων και στα δύο σύνολα. Για κάθε κλάση επιλέγεται ένα ποσοστό των δειγμάτων για επικύρωση, ενώ τα υπόλοιπα χρησιμοποιούνται για εκπαίδευση. Στο τέλος, εφαρμόζεται τυχαία αναδιάταξη των δεδομένων, ώστε να διασφαλιστεί η ομοιόμορφη κατανομή τους.

Η διαδικασία αυτή είναι ιδιαίτερα σημαντική, καθώς επιτρέπει την αξιόπιστη αξιολόγηση των μοντέλων κατά την εκπαίδευση, ενώ παράλληλα δίνει τη δυνατότητα μελέτης της επίδρασης της ισορροπίας των δεδομένων στην απόδοση των αλγορίθμων.

```
# ============================================================
# METRICS
# ============================================================
def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred)


def confusion_matrix_manual(y_true, y_pred, num_classes=10):
    cm = np.zeros((num_classes, num_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def per_class_accuracy(y_true, y_pred, num_classes=10):
    results = {}
    for cls in range(num_classes):
        idx = np.where(y_true == cls)[0]
        cls_acc = np.mean(y_pred[idx] == y_true[idx]) if len(idx) > 0
else 0.0
        results[cls] = cls_acc
    return results


# ============================================================
# PLOTTING
# ============================================================
def save_or_show_plot(filename=None):
    if SAVE_PLOTS and filename is not None:
        plt.savefig(os.path.join(RESULTS_DIR, filename), dpi=300,
bbox_inches="tight")
    if SHOW_PLOTS:
        plt.show()
    plt.close()


def plot_confusion_matrix(cm, title, filename=None):
    plt.figure(figsize=(10, 8))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )
    plt.title(title)
```

```
    plt.xlabel("Predicted class")
    plt.ylabel("True class")
    plt.xticks(rotation=45, ha="right")
    plt.yticks(rotation=0)
    save_or_show_plot(filename)


def plot_training_curve(values, title, ylabel, filename=None):
    plt.figure(figsize=(8, 5))
    plt.plot(range(1, len(values) + 1), values, marker="o")
    plt.title(title)
    plt.xlabel("Epoch")
    plt.ylabel(ylabel)
    plt.grid(True)
    save_or_show_plot(filename)


def plot_examples(X_images, y_true, y_pred, title, correct=True,
num_examples=10, filename=None):
    idx = np.where(y_true == y_pred)[0] if correct else
np.where(y_true != y_pred)[0]

    if len(idx) == 0:
        print(f"No {'correct' if correct else 'incorrect'} examples
found for: {title}")
        return

    chosen = np.random.choice(idx, min(num_examples, len(idx)),
replace=False)

    plt.figure(figsize=(15, 6))
    for i, sample_idx in enumerate(chosen):
        plt.subplot(2, 5, i + 1)
        plt.imshow(X_images[sample_idx], cmap="gray")
        plt.title(
            f"T: {class_names[y_true[sample_idx]]}\nP:
{class_names[y_pred[sample_idx]]}",
            fontsize=9
        )
        plt.axis("off")

    plt.suptitle(title)
    plt.tight_layout()
    save_or_show_plot(filename)
```

Στο επόμενο στάδιο υλοποιούνται οι βασικές μετρικές αξιολόγησης των μοντέλων, καθώς και οι συναρτήσεις οπτικοποίησης των αποτελεσμάτων.

Αρχικά, ορίζεται η συνάρτηση accuracy, η οποία υπολογίζει την ακρίβεια ταξινόμησης, δηλαδή το ποσοστό των σωστά ταξινομημένων δειγμάτων επί του συνόλου. Η ακρίβεια αποτελεί τη βασικότερη μετρική αξιολόγησης και χρησιμοποιείται για τη συνολική εκτίμηση της απόδοσης των μοντέλων.

Στη συνέχεια, υλοποιείται η συνάρτηση confusion_matrix_manual, η οποία κατασκευάζει τον πίνακα σύγχυσης (confusion matrix). Ο πίνακας αυτός απεικονίζει τη σχέση μεταξύ των πραγματικών και των προβλεπόμενων κλάσεων, επιτρέποντας την αναλυτική αξιολόγηση της απόδοσης ανά κατηγορία. Η υλοποίηση γίνεται χειροκίνητα, χωρίς χρήση έτοιμων συναρτήσεων, γεγονός που επιτρέπει καλύτερη κατανόηση της λειτουργίας της μετρικής.

Επιπλέον, ορίζεται η συνάρτηση per_class_accuracy, η οποία υπολογίζει την ακρίβεια για κάθε κλάση ξεχωριστά. Η μετρική αυτή είναι ιδιαίτερα χρήσιμη, ειδικά στην περίπτωση μη ισορροπημένων δεδομένων, καθώς αποκαλύπτει πιθανές αδυναμίες του μοντέλου σε συγκεκριμένες κατηγορίες.

Στο επόμενο τμήμα υλοποιούνται συναρτήσεις που αφορούν την οπτικοποίηση των αποτελεσμάτων. Η συνάρτηση save_or_show_plot διαχειρίζεται την αποθήκευση ή την εμφάνιση των γραφημάτων, ανάλογα με τις ρυθμίσεις του χρήστη. Η συνάρτηση plot_confusion_matrix δημιουργεί οπτική αναπαράσταση του πίνακα σύγχυσης με τη μορφή heatmap, διευκολύνοντας την ερμηνεία των αποτελεσμάτων. Αντίστοιχα, η συνάρτηση plot_training_curve χρησιμοποιείται για την απεικόνιση της εξέλιξης της ακρίβειας ή του σφάλματος κατά τη διάρκεια της εκπαίδευσης.

Τέλος, η συνάρτηση plot_examples χρησιμοποιείται για την οπτική απεικόνιση ενδεικτικών δειγμάτων από το σύνολο δεδομένων, είτε σωστά είτε λανθασμένα ταξινομημένων. Η συνάρτηση αυτή συμβάλλει στην ποιοτική ανάλυση των αποτελεσμάτων, καθώς επιτρέπει την άμεση παρατήρηση των περιπτώσεων όπου το μοντέλο αποδίδει σωστά ή αποτυγχάνει.

Οι παραπάνω μετρικές και τεχνικές οπτικοποίησης αποτελούν βασικά εργαλεία για την ολοκληρωμένη αξιολόγηση των μοντέλων, συνδυάζοντας τόσο ποσοτική όσο και ποιοτική ανάλυση της απόδοσής τους.

```
# ============================================================
# PERCEPTRON OVA
# ============================================================
class PerceptronOVA:
    def __init__(self, lr=0.01, epochs=10):
        self.lr = lr
        self.epochs = epochs
        self.W = None
        self.loss_history = []
        self.train_accuracy_history = []

    def fit(self, X, y):
        num_samples, num_features = X.shape
        self.W = np.zeros((NUM_CLASSES, num_features),
dtype=np.float32)
        self.loss_history = []
        self.train_accuracy_history = []

        for epoch in range(self.epochs):
            errors = 0
            indices = np.random.permutation(num_samples)

            for i in indices:
                xi = X[i]

                for cls in range(NUM_CLASSES):
                    yi = 1 if y[i] == cls else -1
                    if yi * np.dot(self.W[cls], xi) <= 0:
                        self.W[cls] += self.lr * yi * xi
                        errors += 1

            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(errors)
```

```
            self.train_accuracy_history.append(train_acc)

    def predict(self, X):
        scores = X @ self.W.T
        return np.argmax(scores, axis=1)
```

Στο επόμενο στάδιο υλοποιείται ο αλγόριθμος Perceptron στην εκδοχή One-vs-All (OVA), ο οποίος χρησιμοποιείται για την επίλυση προβλημάτων πολυκατηγορικής ταξινόμησης. Στην προσέγγιση αυτή, εκπαιδεύεται ένας δυαδικός ταξινομητής για κάθε κλάση, ο οποίος μαθαίνει να διαχωρίζει τη συγκεκριμένη κλάση από όλες τις υπόλοιπες.

Η υλοποίηση πραγματοποιείται μέσω της κλάσης PerceptronOVA, στην οποία ορίζονται βασικές παράμετροι όπως ο ρυθμός μάθησης (learning rate) και ο αριθμός εποχών εκπαίδευσης (epochs). Επιπλέον, διατηρούνται μεταβλητές για την αποθήκευση των βαρών του μοντέλου, καθώς και ιστορικά δεδομένα της εκπαίδευσης, όπως το πλήθος των σφαλμάτων και η ακρίβεια ανά εποχή.

Κατά τη διαδικασία εκπαίδευσης (fit), αρχικοποιείται ένας πίνακας βαρών, όπου κάθε γραμμή αντιστοιχεί σε έναν ταξινομητή για μία συγκεκριμένη κλάση. Στη συνέχεια, για κάθε εποχή πραγματοποιείται τυχαία αναδιάταξη των δειγμάτων, ώστε να αποφεύγεται η μεροληψία που μπορεί να προκύψει από τη σειρά των δεδομένων.

Για κάθε δείγμα, υπολογίζεται η πρόβλεψη κάθε δυαδικού ταξινομητή, μετατρέποντας το πρόβλημα σε δυαδική μορφή, όπου η επιθυμητή ετικέτα λαμβάνει τιμή +1 για τη σωστή κλάση και -1 για τις υπόλοιπες. Σε περίπτωση λανθασμένης ταξινόμησης, τα βάρη ενημερώνονται σύμφωνα με τον κανόνα του Perceptron, δηλαδή προστίθεται στο διάνυσμα βαρών ένα πολλαπλάσιο του διανύσματος εισόδου, κλιμακωμένο με τον ρυθμό μάθησης και την ετικέτα του δείγματος.

Μετά την ολοκλήρωση κάθε εποχής, υπολογίζεται η ακρίβεια του μοντέλου στο σύνολο εκπαίδευσης, καθώς και ο συνολικός αριθμός των σφαλμάτων, τα οποία αποθηκεύονται για μελλοντική ανάλυση και οπτικοποίηση.

Η διαδικασία πρόβλεψης (predict) βασίζεται στον υπολογισμό των βαθμολογιών (scores) για κάθε κλάση, μέσω του εσωτερικού γινομένου μεταξύ των δεδομένων εισόδου και των αντίστοιχων βαρών. Η τελική πρόβλεψη προκύπτει επιλέγοντας την κλάση με τη μέγιστη τιμή βαθμολογίας.

Η προσέγγιση One-vs-All αποτελεί μια απλή και αποδοτική μέθοδο για την επέκταση του Perceptron σε πολυκατηγορικά προβλήματα, αν και παρουσιάζει περιορισμούς σε περιπτώσεις όπου οι κλάσεις δεν είναι γραμμικά διαχωρίσιμες.

```
# ============================================================
# PERCEPTRON OVO
# ============================================================
class PerceptronOVO:
    def __init__(self, lr=0.01, epochs=5):
        self.lr = lr
        self.epochs = epochs
        self.classifiers = {}
        self.pairs = []
        self.loss_history = []
        self.train_accuracy_history = []

    def fit(self, X, y):
        self.classifiers = {}
```

```
        self.pairs = []
        self.loss_history = []
        self.train_accuracy_history = []

        classes = np.arange(NUM_CLASSES)

        # Prepare pair datasets once
        pair_data = []
        for i in range(len(classes)):
            for j in range(i + 1, len(classes)):
                c1, c2 = classes[i], classes[j]
                idx = np.where((y == c1) | (y == c2))[0]
                X_pair = X[idx]
                y_pair = y[idx]
                y_bin = np.where(y_pair == c1, 1, -1)
                w = np.zeros(X.shape[1], dtype=np.float32)

                self.pairs.append((c1, c2))
                pair_data.append((c1, c2, X_pair, y_bin, w))

        # Global epoch loop so we can keep curves
        for epoch in range(self.epochs):
            total_errors = 0

            new_pair_data = []
            for (c1, c2, X_pair, y_bin, w) in pair_data:
                indices = np.random.permutation(len(y_bin))

                for k in indices:
                    xi = X_pair[k]
                    yi = y_bin[k]
                    if yi * np.dot(w, xi) <= 0:
                        w += self.lr * yi * xi
                        total_errors += 1

                new_pair_data.append((c1, c2, X_pair, y_bin, w))

            pair_data = new_pair_data

            # Store classifiers after each global epoch
            self.classifiers = {(c1, c2): w for (c1, c2, _, _, w) in
pair_data}

            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(total_errors)
            self.train_accuracy_history.append(train_acc)

    def predict(self, X):
        preds = []

        for xi in X:
            votes = np.zeros(NUM_CLASSES, dtype=int)

            for (c1, c2), w in self.classifiers.items():
                score = np.dot(w, xi)
                if score > 0:
                    votes[c1] += 1
                else:
                    votes[c2] += 1
```

```
            preds.append(np.argmax(votes))

        return np.array(preds)
```

Στο επόμενο στάδιο υλοποιείται ο αλγόριθμος Perceptron στην εκδοχή One-vs-One (OVO), ο οποίος αποτελεί μία εναλλακτική προσέγγιση για την επίλυση πολυκατηγορικών προβλημάτων ταξινόμησης. Σε αντίθεση με την προσέγγιση One-vs-All, όπου κάθε ταξινομητής διαχωρίζει μία κλάση από όλες τις υπόλοιπες, στην περίπτωση του One-vs-One εκπαιδεύεται ένας δυαδικός ταξινομητής για κάθε ζεύγος κλάσεων.

Η υλοποίηση πραγματοποιείται μέσω της κλάσης PerceptronOVO, στην οποία ορίζονται βασικές παράμετροι, όπως ο ρυθμός μάθησης και ο αριθμός εποχών εκπαίδευσης. Επιπλέον, χρησιμοποιούνται κατάλληλες δομές δεδομένων για την αποθήκευση των επιμέρους ταξινομητών, καθώς και για την καταγραφή της εξέλιξης της εκπαίδευσης μέσω του πλήθους σφαλμάτων και της ακρίβειας.

Κατά τη διαδικασία εκπαίδευσης (fit), δημιουργούνται αρχικά όλα τα δυνατά ζεύγη κλάσεων. Για κάθε ζεύγος επιλέγονται μόνο τα δείγματα που ανήκουν στις δύο συγκεκριμένες κλάσεις και μετατρέπονται σε δυαδικό πρόβλημα, όπου η μία κλάση αντιστοιχεί σε +1 και η άλλη σε -1. Για κάθε τέτοιο υποσύνολο δεδομένων εκπαιδεύεται ένας ξεχωριστός perceptron ταξινομητής με δικό του διάνυσμα βαρών.

Η εκπαίδευση πραγματοποιείται επαναληπτικά για συγκεκριμένο αριθμό εποχών, όπου σε κάθε εποχή γίνεται τυχαία αναδιάταξη των δειγμάτων και ενημέρωση των βαρών με βάση τον κανόνα του perceptron σε περιπτώσεις λανθασμένης ταξινόμησης. Ταυτόχρονα, καταγράφεται ο συνολικός αριθμός σφαλμάτων για όλα τα ζεύγη ταξινομητών, καθώς και η ακρίβεια στο σύνολο εκπαίδευσης.

Η διαδικασία πρόβλεψης (predict) βασίζεται σε μηχανισμό ψηφοφορίας (voting). Για κάθε δείγμα, κάθε δυαδικός ταξινομητής ψηφίζει υπέρ μίας από τις δύο κλάσεις που διαχωρίζει. Στο τέλος, η τελική πρόβλεψη προκύπτει ως η κλάση που συγκεντρώνει τις περισσότερες ψήφους. Η προσέγγιση αυτή επιτρέπει πιο λεπτομερή διαχωρισμό μεταξύ των κλάσεων, καθώς κάθε ταξινομητής εστιάζει σε ένα συγκεκριμένο υποπρόβλημα.

Η μέθοδος One-vs-One παρουσιάζει συχνά καλύτερη απόδοση σε σχέση με την One-vs-All, ιδιαίτερα σε περιπτώσεις όπου οι κλάσεις παρουσιάζουν υψηλή ομοιότητα. Ωστόσο, το κύριο μειονέκτημά της είναι ο αυξημένος υπολογιστικός φόρτος, καθώς απαιτείται η εκπαίδευση μεγάλου αριθμού ταξινομητών, ο οποίος αυξάνεται τετραγωνικά με τον αριθμό των κλάσεων.

```
# ============================================================
# SOFTMAX CLASSIFIER
# ============================================================
class SoftmaxClassifier:
    def __init__(self, lr=0.01, epochs=100, reg=1e-4,
use_class_weights=False):
        self.lr = lr
        self.epochs = epochs
        self.reg = reg
        self.use_class_weights = use_class_weights
        self.W = None
        self.loss_history = []
        self.train_accuracy_history = []

    def softmax(self, z):
        z = z - np.max(z, axis=1, keepdims=True)  # numerical
stability
        exp_z = np.exp(z)
```

```
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def one_hot(self, y, C):
        y_one = np.zeros((len(y), C), dtype=np.float32)
        y_one[np.arange(len(y)), y] = 1.0
        return y_one

    def compute_class_weights(self, y, C):
        counts = np.bincount(y, minlength=C)
        weights = len(y) / (C * counts + 1e-12)
        return weights.astype(np.float32)

    def fit(self, X, y):
        n, d = X.shape
        C = NUM_CLASSES

        self.W = np.zeros((d, C), dtype=np.float32)
        y_one = self.one_hot(y, C)

        # class weights (optional)
        if self.use_class_weights:
            class_weights = self.compute_class_weights(y, C)
            sample_weights = class_weights[y]
        else:
            sample_weights = np.ones(n, dtype=np.float32)

        self.loss_history = []
        self.train_accuracy_history = []

        for epoch in range(self.epochs):

            indices = np.random.permutation(n)
            X_epoch = X[indices]
            y_one_epoch = y_one[indices]
            sample_weights_epoch = sample_weights[indices]

            # Forward
            scores = X_epoch @ self.W
            probs = self.softmax(scores)

            # Loss (cross-entropy + L2 regularization)
            loss = -np.sum(
                sample_weights_epoch *
                np.sum(y_one_epoch * np.log(probs + 1e-12), axis=1)
            ) / np.sum(sample_weights_epoch)
            loss += self.reg * np.sum(self.W * self.W)

            # Gradient
            grad = (X_epoch.T @ ((probs - y_one_epoch) *
sample_weights_epoch[:, None])) / np.sum(sample_weights_epoch)
            grad += 2 * self.reg * self.W

            # Update
            self.W -= self.lr * grad

            # Metrics
            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(loss)
            self.train_accuracy_history.append(train_acc)
```

```
    def predict(self, X):
        scores = X @ self.W
        probs = self.softmax(scores)
        return np.argmax(probs, axis=1)
```

Στο επόμενο στάδιο υλοποιείται ένας πολυκλασικός γραμμικός ταξινομητής βασισμένος στη μέθοδο της softmax παλινδρόμησης (Softmax Regression). Το μοντέλο αυτό αποτελεί γενίκευση της λογιστικής παλινδρόμησης για προβλήματα πολλαπλών κλάσεων και επιτρέπει την εκτίμηση πιθανοτήτων για κάθε δυνατή κατηγορία.

Η κλάση SoftmaxClassifier υλοποιεί τον ταξινομητή από το μηδέν, χωρίς χρήση έτοιμων βιβλιοθηκών μηχανικής μάθησης. Τα βάρη του μοντέλου αποθηκεύονται σε έναν πίνακα διαστάσεων d×Cd \times Cd×C, όπου ddd είναι ο αριθμός των χαρακτηριστικών και CCC ο αριθμός των κλάσεων.

Η συνάρτηση softmax μετατρέπει τις γραμμικές εξόδους του μοντέλου σε πιθανότητες, εξασφαλίζοντας ότι το άθροισμά τους ισούται με 1. Για λόγους αριθμητικής σταθερότητας, αφαιρείται η μέγιστη τιμή από κάθε διάνυσμα εισόδου πριν την εφαρμογή της εκθετικής συνάρτησης.

$$P(y = c \mid x) = \frac{e^{w_c^T x}}{\sum_{k=1}^{C} e^{w_k^T x}}$$

Για την εκπαίδευση του μοντέλου χρησιμοποιείται η συνάρτηση κόστους cross-entropy, η οποία μετρά την απόκλιση μεταξύ των πραγματικών ετικετών και των προβλεπόμενων πιθανοτήτων. Επιπλέον, εφαρμόζεται κανονικοποίηση L2 (regularization), ώστε να αποφεύγεται η υπερπροσαρμογή (overfitting) και να περιορίζεται το μέγεθος των βαρών.

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(p_{ic}) + \lambda|W|^2$$

Κατά τη διαδικασία εκπαίδευσης εφαρμόζεται gradient descent, όπου τα βάρη ενημερώνονται επαναληπτικά με βάση την κλίση της συνάρτησης κόστους ως προς τα βάρη.

$$W^{(t+1)} = W^{(t)} - \eta\nabla_W L(W)$$

Σημαντική βελτίωση της υλοποίησης αποτελεί η τυχαία αναδιάταξη (shuffling) των δεδομένων σε κάθε εποχή εκπαίδευσης (epoch), γεγονός που συμβάλλει στην αποφυγή μεροληψίας λόγω της σειράς των δεδομένων και οδηγεί σε καλύτερη σύγκλιση του αλγορίθμου.

Επιπλέον, παρέχεται η δυνατότητα χρήσης βαρών κλάσεων (class weights), η οποία είναι ιδιαίτερα χρήσιμη σε περιπτώσεις μη ισορροπημένων συνόλων δεδομένων

(imbalanced datasets). Με τον τρόπο αυτό, οι σπανιότερες κλάσεις αποκτούν μεγαλύτερη σημασία κατά τον υπολογισμό της απώλειας, βελτιώνοντας την απόδοση του μοντέλου σε αυτές.

Τέλος, κατά τη διάρκεια της εκπαίδευσης καταγράφονται η τιμή της συνάρτησης κόστους (loss) και η ακρίβεια (accuracy) στο training set για κάθε epoch, επιτρέποντας την ανάλυση της σύγκλισης και της απόδοσης του μοντέλου. Η πρόβλεψη νέων δειγμάτων πραγματοποιείται επιλέγοντας την κλάση με τη μέγιστη πιθανότητα (argmax) από την έξοδο της softmax συνάρτησης.

```
# ============================================================
# UTILITIES
# ============================================================
def print_per_class_accuracy_table(per_class_acc):
    print("\nPer-class accuracy:")
    for cls, acc in per_class_acc.items():
        print(f"{cls} - {class_names[cls]:12s}: {acc:.4f}")


def get_model_class(model_name):
    if model_name == "Perceptron OVA":
        return PerceptronOVA
    if model_name == "Perceptron OVO":
        return PerceptronOVO
    if model_name == "Softmax":
        return SoftmaxClassifier
    raise ValueError(f"Unknown model name: {model_name}")


def get_hyperparameter_grid(model_name):
    if model_name == "Perceptron OVA":
        return OVA_LRS, OVA_EPOCHS_LIST
    if model_name == "Perceptron OVO":
        return OVO_LRS, OVO_EPOCHS_LIST
    if model_name == "Softmax":
        return SOFTMAX_LRS, SOFTMAX_EPOCHS_LIST
    raise ValueError(f"Unknown model name: {model_name}")


def save_grid_search_results(rows, filename):
    path = os.path.join(RESULTS_DIR, filename)
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=["model", "scenario", "lr", "epochs",
"train_acc", "val_acc"]
        )
        writer.writeheader()
        writer.writerows(rows)


def save_final_results_csv(rows, filename):
    path = os.path.join(RESULTS_DIR, filename)

    fieldnames = ["model", "scenario", "best_lr", "best_epochs",
"test_accuracy"]

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
```

```
        for row in rows:
            filtered_row = {k: row[k] for k in fieldnames}
            writer.writerow(filtered_row)
```

Στο επόμενο στάδιο υλοποιούνται βοηθητικές συναρτήσεις (utilities), οι οποίες υποστηρίζουν τη διαδικασία εκπαίδευσης, αξιολόγησης και αποθήκευσης των αποτελεσμάτων των μοντέλων. Οι συναρτήσεις αυτές δεν επηρεάζουν άμεσα τον τρόπο λειτουργίας των ταξινομητών, αλλά συμβάλλουν στη συστηματική οργάνωση των πειραμάτων και στη διευκόλυνση της ανάλυσης των αποτελεσμάτων.

Αρχικά, η συνάρτηση print_per_class_accuracy_table χρησιμοποιείται για την εκτύπωση της ακρίβειας ανά κλάση (per-class accuracy). Η πληροφορία αυτή είναι ιδιαίτερα σημαντική, καθώς επιτρέπει την αξιολόγηση της απόδοσης του μοντέλου σε κάθε κατηγορία ξεχωριστά, κάτι που είναι κρίσιμο σε περιπτώσεις μη ισορροπημένων δεδομένων.

Στη συνέχεια, οι συναρτήσεις get_model_class και get_hyperparameter_grid παρέχουν έναν δυναμικό μηχανισμό επιλογής μοντέλου και των αντίστοιχων υπερπαραμέτρων του. Με βάση το όνομα του μοντέλου (π.χ. Perceptron OVA, Perceptron OVO, Softmax), επιστρέφεται η αντίστοιχη κλάση υλοποίησης καθώς και το σύνολο τιμών υπερπαραμέτρων (learning rate και αριθμός εποχών) που θα χρησιμοποιηθούν κατά τη διαδικασία αναζήτησης βέλτιστων παραμέτρων (grid search). Με τον τρόπο αυτό αποφεύγεται η επανάληψη κώδικα και επιτυγχάνεται μεγαλύτερη επεκτασιμότητα.

Επιπλέον, υλοποιούνται συναρτήσεις για την αποθήκευση των αποτελεσμάτων σε αρχεία CSV. Η συνάρτηση save_grid_search_results αποθηκεύει τα αποτελέσματα της διαδικασίας grid search, συμπεριλαμβανομένων των τιμών των υπερπαραμέτρων και της απόδοσης σε training και validation σύνολα. Αντίστοιχα, η συνάρτηση save_final_results_csv αποθηκεύει τα τελικά αποτελέσματα των βέλτιστων μοντέλων, καταγράφοντας τις επιλεγμένες υπερπαραμέτρους και την ακρίβεια στο test set.

Η αποθήκευση των αποτελεσμάτων σε δομημένη μορφή επιτρέπει την εύκολη περαιτέρω ανάλυση, την αναπαραγωγιμότητα των πειραμάτων και τη δημιουργία γραφημάτων ή συγκριτικών πινάκων. Συνολικά, οι βοηθητικές αυτές συναρτήσεις αποτελούν σημαντικό μέρος της πειραματικής διαδικασίας, ενισχύοντας την οργάνωση, την ευελιξία και την αξιοπιστία της υλοποίησης.

```
# ============================================================
# MODEL SELECTION
# ============================================================
def hyperparameter_search(model_name, scenario_name, X_train,
y_train, X_val, y_val):
    ModelClass = get_model_class(model_name)
    lrs, epoch_list = get_hyperparameter_grid(model_name)

    best_val_acc = -1.0
    best_lr = None
    best_epochs = None
    search_rows = []

    print("\n" + "-" * 70)
    print(f"Hyperparameter Search | {model_name} | {scenario_name}")
    print("-" * 70)

    for lr in lrs:
        for epochs in epoch_list:
```

```
            if model_name == "Softmax":
                use_weights = (scenario_name == "Imbalanced")
                model = ModelClass(lr=lr, epochs=epochs,
use_class_weights=use_weights)
            else:
                model = ModelClass(lr=lr, epochs=epochs)
            model.fit(X_train, y_train)

            train_pred = model.predict(X_train)
            val_pred = model.predict(X_val)

            train_acc = accuracy(y_train, train_pred)
            val_acc = accuracy(y_val, val_pred)

            print(
                f"{model_name:15s} | {scenario_name:10s} | "
                f"lr={lr:<6} | epochs={epochs:<3d} | "
                f"train={train_acc:.4f} | val={val_acc:.4f}"
            )

            search_rows.append({
                "model": model_name,
                "scenario": scenario_name,
                "lr": lr,
                "epochs": epochs,
                "train_acc": train_acc,
                "val_acc": val_acc
            })

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_lr = lr
                best_epochs = epochs

    return best_lr, best_epochs, best_val_acc, search_rows
```

Στο επόμενο στάδιο υλοποιείται η διαδικασία επιλογής μοντέλου (model selection), η οποία βασίζεται στη δοκιμή διαφορετικών συνδυασμών υπερπαραμέτρων, με στόχο την εύρεση των βέλτιστων ρυθμίσεων για κάθε αλγόριθμο.

Η διαδικασία αυτή υλοποιείται μέσω της συνάρτησης hyperparameter_search, η οποία δέχεται ως είσοδο το όνομα του μοντέλου, το σενάριο δεδομένων (balanced ή imbalanced), καθώς και τα σύνολα εκπαίδευσης και επικύρωσης. Αρχικά, προσδιορίζεται δυναμικά η αντίστοιχη κλάση του μοντέλου και το σύνολο των υπερπαραμέτρων που θα εξεταστούν, όπως ο ρυθμός μάθησης και ο αριθμός εποχών εκπαίδευσης.

Στη συνέχεια, εκτελείται διαδικασία αναζήτησης πλέγματος (grid search), κατά την οποία δοκιμάζονται όλοι οι δυνατοί συνδυασμοί των υπερπαραμέτρων. Για κάθε συνδυασμό, δημιουργείται ένα νέο μοντέλο, το οποίο εκπαιδεύεται στο σύνολο εκπαίδευσης και αξιολογείται τόσο στο training όσο και στο validation set.

Για κάθε εκτέλεση υπολογίζεται η ακρίβεια ταξινόμησης, ενώ τα αποτελέσματα καταγράφονται και αποθηκεύονται σε κατάλληλη δομή δεδομένων. Παράλληλα, εμφανίζονται αναλυτικά στοιχεία στην οθόνη, διευκολύνοντας την παρακολούθηση της διαδικασίας.

Κατά τη διάρκεια της αναζήτησης, διατηρείται ο συνδυασμός υπερπαραμέτρων που επιτυγχάνει τη μεγαλύτερη ακρίβεια στο σύνολο επικύρωσης. Η επιλογή αυτή

βασίζεται στο validation set, ώστε να αποφευχθεί η υπερπροσαρμογή (overfitting) στο training set και να εξασφαλιστεί καλύτερη γενίκευση του μοντέλου.

Τέλος, η συνάρτηση επιστρέφει τις βέλτιστες τιμές των υπερπαραμέτρων, την αντίστοιχη ακρίβεια επικύρωσης, καθώς και όλα τα ενδιάμεσα αποτελέσματα της διαδικασίας. Η προσέγγιση αυτή επιτρέπει τη συστηματική και αντικειμενική επιλογή των παραμέτρων των μοντέλων, συμβάλλοντας στη βελτίωση της συνολικής τους απόδοσης.

```
# ============================================================
# FINAL EVALUATION
# ============================================================
def evaluate_best_model(
    model_name,
    scenario_name,
    X_train_used,
    y_train_used,
    X_test,
    y_test,
    X_test_images,
    best_lr,
    best_epochs
):
    print("\n" + "=" * 70)
    print(f"FINAL EVALUATION | {model_name} | {scenario_name}")
    print("=" * 70)
    print(f"Best hyperparameters: lr={best_lr},
epochs={best_epochs}")

    ModelClass = get_model_class(model_name)

    if model_name == "Softmax":
        use_weights = (scenario_name == "Imbalanced")
        model = ModelClass(
            lr=best_lr,
            epochs=best_epochs,
            use_class_weights=use_weights
        )
    else:
        model = ModelClass(lr=best_lr, epochs=best_epochs)
    model.fit(X_train_used, y_train_used)

    y_pred = model.predict(X_test)

    acc = accuracy(y_test, y_pred)
    cm = confusion_matrix_manual(y_test, y_pred, NUM_CLASSES)
    pca = per_class_accuracy(y_test, y_pred, NUM_CLASSES)

    print(f"Test Accuracy: {acc:.4f}")
    print_per_class_accuracy_table(pca)

    base_name = f"{model_name}_{scenario_name}".replace(" ", "_")

    plot_confusion_matrix(
        cm,
        title=f"{model_name} - {scenario_name}",
        filename=f"{base_name}_confusion_matrix.png"
    )

    if hasattr(model, "loss_history") and len(model.loss_history) >
```

```
0:
        y_label = "Errors" if "Perceptron" in model_name else "Loss"
        plot_training_curve(
            model.loss_history,
            title=f"{model_name} - {scenario_name} Training Curve",
            ylabel=y_label,
            filename=f"{base_name}_training_curve.png"
        )

    if hasattr(model, "train_accuracy_history") and
len(model.train_accuracy_history) > 0:
        plot_training_curve(
            model.train_accuracy_history,
            title=f"{model_name} - {scenario_name} Train Accuracy",
            ylabel="Accuracy",
            filename=f"{base_name}_train_accuracy.png"
        )

    plot_examples(
        X_test_images,
        y_test,
        y_pred,
        title=f"{model_name} - {scenario_name} | Correct
Classifications",
        correct=True,
        num_examples=10,
        filename=f"{base_name}_correct_examples.png"
    )

    plot_examples(
        X_test_images,
        y_test,
        y_pred,
        title=f"{model_name} - {scenario_name} | Incorrect
Classifications",
        correct=False,
        num_examples=10,
        filename=f"{base_name}_incorrect_examples.png"
    )

    return {
        "model": model_name,
        "scenario": scenario_name,
        "best_lr": best_lr,
        "best_epochs": best_epochs,
        "test_accuracy": acc,
        "per_class_accuracy": pca,
        "confusion_matrix": cm
    }
```

Στο τελικό στάδιο υλοποιείται η διαδικασία αξιολόγησης του βέλτιστου μοντέλου, η οποία βασίζεται στις υπερπαραμέτρους που προέκυψαν από τη διαδικασία επιλογής μοντέλου. Η αξιολόγηση αυτή πραγματοποιείται μέσω της συνάρτησης evaluate_best_model, η οποία εφαρμόζεται ξεχωριστά για κάθε μοντέλο και για κάθε σενάριο δεδομένων (balanced ή imbalanced).

Αρχικά, δημιουργείται ένα νέο μοντέλο χρησιμοποιώντας τις βέλτιστες τιμές του ρυθμού μάθησης και του αριθμού εποχών εκπαίδευσης. Το μοντέλο εκπαιδεύεται εκ

νέου στο πλήρες σύνολο εκπαίδευσης που έχει χρησιμοποιηθεί, ώστε να αξιοποιηθεί στο μέγιστο η διαθέσιμη πληροφορία.

Στη συνέχεια, πραγματοποιείται πρόβλεψη στο σύνολο ελέγχου (test set) και υπολογίζονται βασικές μετρικές αξιολόγησης. Συγκεκριμένα, υπολογίζεται η συνολική ακρίβεια ταξινόμησης, ο πίνακας σύγχυσης, καθώς και η ακρίβεια ανά κλάση. Οι μετρικές αυτές παρέχουν τόσο συνολική όσο και αναλυτική εικόνα της απόδοσης του μοντέλου.

Παράλληλα, πραγματοποιείται εκτενής οπτικοποίηση των αποτελεσμάτων. Ο πίνακας σύγχυσης απεικονίζεται σε μορφή heatmap, επιτρέποντας την εύκολη αναγνώριση των σωστών και λανθασμένων ταξινομήσεων ανά κατηγορία. Επιπλέον, εφόσον είναι διαθέσιμα δεδομένα εκπαίδευσης, απεικονίζονται οι καμπύλες εξέλιξης του σφάλματος (loss ή errors) και της ακρίβειας κατά τη διάρκεια των εποχών, παρέχοντας πληροφορία σχετικά με τη σύγκλιση του μοντέλου.

Τέλος, πραγματοποιείται οπτική απεικόνιση ενδεικτικών δειγμάτων από το σύνολο ελέγχου, τόσο σωστά όσο και λανθασμένα ταξινομημένων. Η διαδικασία αυτή είναι ιδιαίτερα σημαντική για την ποιοτική ανάλυση των αποτελεσμάτων, καθώς επιτρέπει την κατανόηση των περιπτώσεων στις οποίες το μοντέλο αποδίδει σωστά ή παρουσιάζει σφάλματα.

Η συνάρτηση επιστρέφει ένα σύνολο αποτελεσμάτων που περιλαμβάνει τις βέλτιστες υπερπαραμέτρους, την τελική ακρίβεια στο test set, καθώς και αναλυτικές πληροφορίες όπως ο πίνακας σύγχυσης και η ακρίβεια ανά κλάση. Η διαδικασία αυτή ολοκληρώνει το πειραματικό πλαίσιο, παρέχοντας μια πλήρη και αξιόπιστη αξιολόγηση της απόδοσης των μοντέλων.

```
# ============================================================
# MAIN
# ============================================================
def main():
    print("Creating balanced and imbalanced datasets...")

    X_bal, y_bal = create_balanced_dataset(
        X_train_full, y_train_full,
        samples_per_class=BALANCED_SAMPLES_PER_CLASS
    )

    X_imb, y_imb = create_imbalanced_dataset(
        X_train_full, y_train_full,
        main_class=IMBALANCED_MAIN_CLASS,
        main_samples=IMBALANCED_MAIN_SAMPLES,
        other_samples=IMBALANCED_OTHER_SAMPLES
    )

    print("\nBalanced distribution:")
    print(Counter(y_bal))

    print("\nImbalanced distribution:")
    print(Counter(y_imb))
    print(f"\nMain class in imbalanced scenario:
{IMBALANCED_MAIN_CLASS} ({class_names[IMBALANCED_MAIN_CLASS]})")

    scenarios = [
        ("Balanced", X_bal, y_bal),
        ("Imbalanced", X_imb, y_imb)
    ]
```

```
    model_names = [
        "Perceptron OVA",
        "Perceptron OVO",
        "Softmax"
    ]

    all_search_rows = []
    final_results = []

    for scenario_name, X_scenario, y_scenario in scenarios:
        X_train_split, y_train_split, X_val_split, y_val_split =
stratified_train_val_split(
            X_scenario, y_scenario, val_ratio=VALIDATION_RATIO
        )

        for model_name in model_names:
            best_lr, best_epochs, best_val_acc, search_rows =
hyperparameter_search(
                model_name=model_name,
                scenario_name=scenario_name,
                X_train=X_train_split,
                y_train=y_train_split,
                X_val=X_val_split,
                y_val=y_val_split
            )

            all_search_rows.extend(search_rows)

            print(
                f"\nBest for {model_name} | {scenario_name}: "
                f"lr={best_lr}, epochs={best_epochs},
val_acc={best_val_acc:.4f}"
            )

            # Retrain on full scenario training data with best
hyperparameters
            final_result = evaluate_best_model(
                model_name=model_name,
                scenario_name=scenario_name,
                X_train_used=X_scenario,
                y_train_used=y_scenario,
                X_test=X_test,
                y_test=y_test,
                X_test_images=X_test_images,
                best_lr=best_lr,
                best_epochs=best_epochs
            )

            final_results.append(final_result)

    # Save grid search results
    save_grid_search_results(all_search_rows,
"grid_search_results.csv")

    # Save final results CSV
    save_final_results_csv(final_results, "final_results.csv")

    # Final summary
    print("\n" + "=" * 70)
    print("FINAL SUMMARY")
```

```
    print("=" * 70)
    for res in final_results:
        print(
            f"{res['model']:15s} | {res['scenario']:10s} | "
            f"lr={res['best_lr']:<6} |
epochs={res['best_epochs']:<3d} | "
            f"Test Accuracy = {res['test_accuracy']:.4f}"
        )

    summary_path = os.path.join(RESULTS_DIR, "summary_results.txt")
    with open(summary_path, "w", encoding="utf-8") as f:
        f.write("FINAL SUMMARY\n")
        f.write("=" * 70 + "\n")
        for res in final_results:
            f.write(
                f"{res['model']:15s} | {res['scenario']:10s} | "
                f"lr={res['best_lr']:<6} |
epochs={res['best_epochs']:<3d} | "
                f"Test Accuracy = {res['test_accuracy']:.4f}\n"
            )

    print(f"\nAll results saved in folder: {RESULTS_DIR}")


if __name__ == "__main__":
    main()
```

Στο τελικό στάδιο υλοποιείται η κύρια συνάρτηση του προγράμματος (main), η οποία οργανώνει και εκτελεί ολόκληρη τη διαδικασία των πειραμάτων, από τη δημιουργία των δεδομένων έως την τελική αξιολόγηση των μοντέλων.

Αρχικά, δημιουργούνται δύο διαφορετικά σύνολα δεδομένων: ένα ισορροπημένο (balanced) και ένα μη ισορροπημένο (imbalanced). Στο ισορροπημένο σύνολο, όλες οι κλάσεις περιέχουν τον ίδιο αριθμό δειγμάτων, ενώ στο μη ισορροπημένο σύνολο ορίζεται μία κυρίαρχη κλάση με σημαντικά μεγαλύτερο πλήθος παραδειγμάτων σε σχέση με τις υπόλοιπες. Στη συνέχεια, εμφανίζεται η κατανομή των κλάσεων για κάθε σενάριο, ώστε να επιβεβαιωθεί η σωστή διαμόρφωση των δεδομένων.

Έπειτα, ορίζονται τα σενάρια πειραμάτων (balanced και imbalanced) καθώς και τα μοντέλα που θα αξιολογηθούν, δηλαδή τα Perceptron OVA, Perceptron OVO και Softmax Regression. Για κάθε συνδυασμό σεναρίου και μοντέλου, πραγματοποιείται διαχωρισμός των δεδομένων σε σύνολο εκπαίδευσης και επικύρωσης με στρωματοποιημένο τρόπο, ώστε να διατηρείται η αναλογία των κλάσεων.

Στη συνέχεια, εφαρμόζεται η διαδικασία αναζήτησης υπερπαραμέτρων (grid search), κατά την οποία δοκιμάζονται διαφορετικοί συνδυασμοί ρυθμού μάθησης και αριθμού εποχών. Για κάθε συνδυασμό υπολογίζεται η απόδοση στο σύνολο επικύρωσης, και επιλέγονται οι τιμές που μεγιστοποιούν την ακρίβεια. Τα αποτελέσματα της διαδικασίας αυτής αποθηκεύονται για περαιτέρω ανάλυση.

Αφού προσδιοριστούν οι βέλτιστες υπερπαράμετροι, το μοντέλο επανεκπαιδεύεται στο πλήρες σύνολο εκπαίδευσης του αντίστοιχου σεναρίου και αξιολογείται στο σύνολο ελέγχου. Κατά την αξιολόγηση υπολογίζονται βασικές μετρικές, όπως η συνολική ακρίβεια, η ακρίβεια ανά κλάση και ο πίνακας σύγχυσης, ενώ παράλληλα παράγονται και τα αντίστοιχα γραφήματα και οπτικοποιήσεις.

Όλα τα αποτελέσματα των πειραμάτων αποθηκεύονται σε αρχεία CSV, ενώ δημιουργείται και ένα συνοπτικό αρχείο κειμένου που περιλαμβάνει τα τελικά

αποτελέσματα για κάθε μοντέλο και σενάριο. Τέλος, εμφανίζεται στην οθόνη μία συνοπτική παρουσίαση των αποτελεσμάτων, διευκολύνοντας την άμεση σύγκριση των μοντέλων.

Η συνάρτηση main αποτελεί το κεντρικό σημείο ελέγχου της εφαρμογής, συντονίζοντας όλα τα επιμέρους στάδια της διαδικασίας και διασφαλίζοντας την ομαλή εκτέλεση των πειραμάτων, καθώς και την οργανωμένη καταγραφή των αποτελεσμάτων.

8. Παράρτημα – Πλήρης Κώδικας

```
# ============================================================
# LINEAR CLASSIFIERS PROJECT - FROM SCRATCH
# Improved version to better cover the full assignment:
# - Perceptron One-vs-All (OVA)
# - Perceptron One-vs-One (OVO)
# - Softmax Regression
# - Author: Grigoris Liatsos
# ============================================================

import os
import csv
import random
from collections import Counter

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from keras.datasets import fashion_mnist


# ============================================================
# REPRODUCIBILITY
# ============================================================
SEED = 42
random.seed(SEED)
np.random.seed(SEED)


# ============================================================
# CONFIG
# ============================================================
RESULTS_DIR = "results_project2_full"
os.makedirs(RESULTS_DIR, exist_ok=True)

NUM_CLASSES = 10
BALANCED_SAMPLES_PER_CLASS = 1000
IMBALANCED_MAIN_CLASS = 0
IMBALANCED_MAIN_SAMPLES = 1000
IMBALANCED_OTHER_SAMPLES = 50

VALIDATION_RATIO = 0.2

# Hyperparameter grids
OVA_LRS = [0.001, 0.01, 0.1]
OVA_EPOCHS_LIST = [5, 10, 20]
```

```
OVO_LRS = [0.001, 0.01, 0.1]
OVO_EPOCHS_LIST = [3, 5, 10]

SOFTMAX_LRS = [0.0005, 0.001, 0.003]
SOFTMAX_EPOCHS_LIST = [100, 200, 300]

SHOW_PLOTS = False
SAVE_PLOTS = True


# ============================================================
# CLASS NAMES
# ============================================================
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]


# ============================================================
# LOAD DATASET
# ============================================================
(X_train_full_raw, y_train_full), (X_test_raw, y_test) =
fashion_mnist.load_data()

# Keep original images for visualization
X_test_images = X_test_raw.copy()

# Flatten + normalize
X_train_full = X_train_full_raw.reshape(X_train_full_raw.shape[0], -
1).astype(np.float32) / 255.0
X_test = X_test_raw.reshape(X_test_raw.shape[0], -
1).astype(np.float32) / 255.0

# ============================================================
# STANDARDIZATION
# ============================================================
mean = np.mean(X_train_full, axis=0)
std = np.std(X_train_full, axis=0) + 1e-8

X_train_full = (X_train_full - mean) / std
X_test = (X_test - mean) / std

# ============================================================
# ADD BIAS FEATURE
# ============================================================
def add_bias_feature(X):
    return np.hstack([X, np.ones((X.shape[0], 1), dtype=X.dtype)])

X_train_full = add_bias_feature(X_train_full)
X_test = add_bias_feature(X_test)


# ============================================================
# DATASET CREATION
# ============================================================
def create_balanced_dataset(X, y, samples_per_class=1000):
    X_bal, y_bal = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
```

```
        chosen = np.random.choice(idx, samples_per_class,
replace=False)
        X_bal.append(X[chosen])
        y_bal.append(y[chosen])

    X_bal = np.vstack(X_bal)
    y_bal = np.hstack(y_bal)

    perm = np.random.permutation(len(y_bal))
    return X_bal[perm], y_bal[perm]


def create_imbalanced_dataset(X, y, main_class=0, main_samples=1000,
other_samples=50):
    X_imb, y_imb = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
        if cls == main_class:
            chosen = np.random.choice(idx, main_samples,
replace=False)
        else:
            chosen = np.random.choice(idx, other_samples,
replace=False)

        X_imb.append(X[chosen])
        y_imb.append(y[chosen])

    X_imb = np.vstack(X_imb)
    y_imb = np.hstack(y_imb)

    perm = np.random.permutation(len(y_imb))
    return X_imb[perm], y_imb[perm]


def stratified_train_val_split(X, y, val_ratio=0.2):
    X_train_parts, y_train_parts = [], []
    X_val_parts, y_val_parts = [], []

    for cls in range(NUM_CLASSES):
        idx = np.where(y == cls)[0]
        idx = np.random.permutation(idx)

        n_val = max(1, int(len(idx) * val_ratio))
        val_idx = idx[:n_val]
        train_idx = idx[n_val:]

        X_val_parts.append(X[val_idx])
        y_val_parts.append(y[val_idx])

        X_train_parts.append(X[train_idx])
        y_train_parts.append(y[train_idx])

    X_train_split = np.vstack(X_train_parts)
    y_train_split = np.hstack(y_train_parts)
    X_val_split = np.vstack(X_val_parts)
    y_val_split = np.hstack(y_val_parts)

    train_perm = np.random.permutation(len(y_train_split))
    val_perm = np.random.permutation(len(y_val_split))
```

```
    return (
        X_train_split[train_perm], y_train_split[train_perm],
        X_val_split[val_perm], y_val_split[val_perm]
    )


# ============================================================
# METRICS
# ============================================================
def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred)


def confusion_matrix_manual(y_true, y_pred, num_classes=10):
    cm = np.zeros((num_classes, num_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def per_class_accuracy(y_true, y_pred, num_classes=10):
    results = {}
    for cls in range(num_classes):
        idx = np.where(y_true == cls)[0]
        cls_acc = np.mean(y_pred[idx] == y_true[idx]) if len(idx) > 0
else 0.0
        results[cls] = cls_acc
    return results


# ============================================================
# PLOTTING
# ============================================================
def save_or_show_plot(filename=None):
    if SAVE_PLOTS and filename is not None:
        plt.savefig(os.path.join(RESULTS_DIR, filename), dpi=300,
bbox_inches="tight")
    if SHOW_PLOTS:
        plt.show()
    plt.close()


def plot_confusion_matrix(cm, title, filename=None):
    plt.figure(figsize=(10, 8))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )
    plt.title(title)
    plt.xlabel("Predicted class")
    plt.ylabel("True class")
    plt.xticks(rotation=45, ha="right")
    plt.yticks(rotation=0)
    save_or_show_plot(filename)


def plot_training_curve(values, title, ylabel, filename=None):
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(range(1, len(values) + 1), values, marker="o")
    plt.title(title)
    plt.xlabel("Epoch")
    plt.ylabel(ylabel)
    plt.grid(True)
    save_or_show_plot(filename)


def plot_examples(X_images, y_true, y_pred, title, correct=True,
num_examples=10, filename=None):
    idx = np.where(y_true == y_pred)[0] if correct else
np.where(y_true != y_pred)[0]

    if len(idx) == 0:
        print(f"No {'correct' if correct else 'incorrect'} examples
found for: {title}")
        return

    chosen = np.random.choice(idx, min(num_examples, len(idx)),
replace=False)

    plt.figure(figsize=(15, 6))
    for i, sample_idx in enumerate(chosen):
        plt.subplot(2, 5, i + 1)
        plt.imshow(X_images[sample_idx], cmap="gray")
        plt.title(
            f"T: {class_names[y_true[sample_idx]]}\nP:
{class_names[y_pred[sample_idx]]}",
            fontsize=9
        )
        plt.axis("off")

    plt.suptitle(title)
    plt.tight_layout()
    save_or_show_plot(filename)


# ============================================================
# PERCEPTRON OVA
# ============================================================
class PerceptronOVA:
    def __init__(self, lr=0.01, epochs=10):
        self.lr = lr
        self.epochs = epochs
        self.W = None
        self.loss_history = []
        self.train_accuracy_history = []

    def fit(self, X, y):
        num_samples, num_features = X.shape
        self.W = np.zeros((NUM_CLASSES, num_features),
dtype=np.float32)
        self.loss_history = []
        self.train_accuracy_history = []

        for epoch in range(self.epochs):
            errors = 0
            indices = np.random.permutation(num_samples)

            for i in indices:
```

```
                xi = X[i]

                for cls in range(NUM_CLASSES):
                    yi = 1 if y[i] == cls else -1
                    if yi * np.dot(self.W[cls], xi) <= 0:
                        self.W[cls] += self.lr * yi * xi
                        errors += 1

            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(errors)
            self.train_accuracy_history.append(train_acc)

    def predict(self, X):
        scores = X @ self.W.T
        return np.argmax(scores, axis=1)


# ============================================================
# PERCEPTRON OVO
# ============================================================
class PerceptronOVO:
    def __init__(self, lr=0.01, epochs=5):
        self.lr = lr
        self.epochs = epochs
        self.classifiers = {}
        self.pairs = []
        self.loss_history = []
        self.train_accuracy_history = []

    def fit(self, X, y):
        self.classifiers = {}
        self.pairs = []
        self.loss_history = []
        self.train_accuracy_history = []

        classes = np.arange(NUM_CLASSES)

        # Prepare pair datasets once
        pair_data = []
        for i in range(len(classes)):
            for j in range(i + 1, len(classes)):
                c1, c2 = classes[i], classes[j]
                idx = np.where((y == c1) | (y == c2))[0]
                X_pair = X[idx]
                y_pair = y[idx]
                y_bin = np.where(y_pair == c1, 1, -1)
                w = np.zeros(X.shape[1], dtype=np.float32)

                self.pairs.append((c1, c2))
                pair_data.append((c1, c2, X_pair, y_bin, w))

        # Global epoch loop so we can keep curves
        for epoch in range(self.epochs):
            total_errors = 0

            new_pair_data = []
            for (c1, c2, X_pair, y_bin, w) in pair_data:
                indices = np.random.permutation(len(y_bin))
```

```
                for k in indices:
                    xi = X_pair[k]
                    yi = y_bin[k]
                    if yi * np.dot(w, xi) <= 0:
                        w += self.lr * yi * xi
                        total_errors += 1

                new_pair_data.append((c1, c2, X_pair, y_bin, w))

            pair_data = new_pair_data

            # Store classifiers after each global epoch
            self.classifiers = {(c1, c2): w for (c1, c2, _, _, w) in
pair_data}

            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(total_errors)
            self.train_accuracy_history.append(train_acc)

    def predict(self, X):
        preds = []

        for xi in X:
            votes = np.zeros(NUM_CLASSES, dtype=int)

            for (c1, c2), w in self.classifiers.items():
                score = np.dot(w, xi)
                if score > 0:
                    votes[c1] += 1
                else:
                    votes[c2] += 1

            preds.append(np.argmax(votes))

        return np.array(preds)


# ============================================================
# SOFTMAX CLASSIFIER
# ============================================================
class SoftmaxClassifier:
    def __init__(self, lr=0.01, epochs=100, reg=1e-4,
use_class_weights=False):
        self.lr = lr
        self.epochs = epochs
        self.reg = reg
        self.use_class_weights = use_class_weights
        self.W = None
        self.loss_history = []
        self.train_accuracy_history = []

    def softmax(self, z):
        z = z - np.max(z, axis=1, keepdims=True)  # numerical
stability
        exp_z = np.exp(z)
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def one_hot(self, y, C):
        y_one = np.zeros((len(y), C), dtype=np.float32)
```

```
        y_one[np.arange(len(y)), y] = 1.0
        return y_one

    def compute_class_weights(self, y, C):
        counts = np.bincount(y, minlength=C)
        weights = len(y) / (C * counts + 1e-12)
        return weights.astype(np.float32)

    def fit(self, X, y):
        n, d = X.shape
        C = NUM_CLASSES

        self.W = np.zeros((d, C), dtype=np.float32)
        y_one = self.one_hot(y, C)

        # class weights
        if self.use_class_weights:
            class_weights = self.compute_class_weights(y, C)
            sample_weights = class_weights[y]
        else:
            sample_weights = np.ones(n, dtype=np.float32)

        self.loss_history = []
        self.train_accuracy_history = []

        for epoch in range(self.epochs):
            # SHUFFLE κάθε epoch
            indices = np.random.permutation(n)
            X_epoch = X[indices]
            y_one_epoch = y_one[indices]
            sample_weights_epoch = sample_weights[indices]

            # Forward
            scores = X_epoch @ self.W
            probs = self.softmax(scores)

            # Loss (cross-entropy + L2 regularization)
            loss = -np.sum(
                sample_weights_epoch *
                np.sum(y_one_epoch * np.log(probs + 1e-12), axis=1)
            ) / np.sum(sample_weights_epoch)
            loss += self.reg * np.sum(self.W * self.W)

            # Gradient
            grad = (X_epoch.T @ ((probs - y_one_epoch) *
sample_weights_epoch[:, None])) / np.sum(sample_weights_epoch)
            grad += 2 * self.reg * self.W

            # Update
            self.W -= self.lr * grad

            # Metrics
            preds_train = self.predict(X)
            train_acc = accuracy(y, preds_train)

            self.loss_history.append(loss)
            self.train_accuracy_history.append(train_acc)

    def predict(self, X):
        scores = X @ self.W
        probs = self.softmax(scores)
```

```
        return np.argmax(probs, axis=1)

# ============================================================
# UTILITIES
# ============================================================
def print_per_class_accuracy_table(per_class_acc):
    print("\nPer-class accuracy:")
    for cls, acc in per_class_acc.items():
        print(f"{cls} - {class_names[cls]:12s}: {acc:.4f}")


def get_model_class(model_name):
    if model_name == "Perceptron OVA":
        return PerceptronOVA
    if model_name == "Perceptron OVO":
        return PerceptronOVO
    if model_name == "Softmax":
        return SoftmaxClassifier
    raise ValueError(f"Unknown model name: {model_name}")


def get_hyperparameter_grid(model_name):
    if model_name == "Perceptron OVA":
        return OVA_LRS, OVA_EPOCHS_LIST
    if model_name == "Perceptron OVO":
        return OVO_LRS, OVO_EPOCHS_LIST
    if model_name == "Softmax":
        return SOFTMAX_LRS, SOFTMAX_EPOCHS_LIST
    raise ValueError(f"Unknown model name: {model_name}")


def save_grid_search_results(rows, filename):
    path = os.path.join(RESULTS_DIR, filename)
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=["model", "scenario", "lr", "epochs",
"train_acc", "val_acc"]
        )
        writer.writeheader()
        writer.writerows(rows)


def save_final_results_csv(rows, filename):
    path = os.path.join(RESULTS_DIR, filename)

    fieldnames = ["model", "scenario", "best_lr", "best_epochs",
"test_accuracy"]

    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()

        for row in rows:
            filtered_row = {k: row[k] for k in fieldnames}
            writer.writerow(filtered_row)


# ============================================================
# MODEL SELECTION
# ============================================================
```

```
def hyperparameter_search(model_name, scenario_name, X_train,
y_train, X_val, y_val):
    ModelClass = get_model_class(model_name)
    lrs, epoch_list = get_hyperparameter_grid(model_name)

    best_val_acc = -1.0
    best_lr = None
    best_epochs = None
    search_rows = []

    print("\n" + "-" * 70)
    print(f"Hyperparameter Search | {model_name} | {scenario_name}")
    print("-" * 70)

    for lr in lrs:
        for epochs in epoch_list:
            if model_name == "Softmax":
                use_weights = (scenario_name == "Imbalanced")
                model = ModelClass(lr=lr, epochs=epochs,
use_class_weights=use_weights)
            else:
                model = ModelClass(lr=lr, epochs=epochs)
            model.fit(X_train, y_train)

            train_pred = model.predict(X_train)
            val_pred = model.predict(X_val)

            train_acc = accuracy(y_train, train_pred)
            val_acc = accuracy(y_val, val_pred)

            print(
                f"{model_name:15s} | {scenario_name:10s} | "
                f"lr={lr:<6} | epochs={epochs:<3d} | "
                f"train={train_acc:.4f} | val={val_acc:.4f}"
            )

            search_rows.append({
                "model": model_name,
                "scenario": scenario_name,
                "lr": lr,
                "epochs": epochs,
                "train_acc": train_acc,
                "val_acc": val_acc
            })

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_lr = lr
                best_epochs = epochs

    return best_lr, best_epochs, best_val_acc, search_rows


# ============================================================
# FINAL EVALUATION
# ============================================================
def evaluate_best_model(
    model_name,
    scenario_name,
    X_train_used,
    y_train_used,
```

```
    X_test,
    y_test,
    X_test_images,
    best_lr,
    best_epochs
):
    print("\n" + "=" * 70)
    print(f"FINAL EVALUATION | {model_name} | {scenario_name}")
    print("=" * 70)
    print(f"Best hyperparameters: lr={best_lr},
epochs={best_epochs}")

    ModelClass = get_model_class(model_name)

    if model_name == "Softmax":
        use_weights = (scenario_name == "Imbalanced")
        model = ModelClass(
            lr=best_lr,
            epochs=best_epochs,
            use_class_weights=use_weights
        )
    else:
        model = ModelClass(lr=best_lr, epochs=best_epochs)
    model.fit(X_train_used, y_train_used)

    y_pred = model.predict(X_test)

    acc = accuracy(y_test, y_pred)
    cm = confusion_matrix_manual(y_test, y_pred, NUM_CLASSES)
    pca = per_class_accuracy(y_test, y_pred, NUM_CLASSES)

    print(f"Test Accuracy: {acc:.4f}")
    print_per_class_accuracy_table(pca)

    base_name = f"{model_name}_{scenario_name}".replace(" ", "_")

    plot_confusion_matrix(
        cm,
        title=f"{model_name} - {scenario_name}",
        filename=f"{base_name}_confusion_matrix.png"
    )

    if hasattr(model, "loss_history") and len(model.loss_history) >
0:
        y_label = "Errors" if "Perceptron" in model_name else "Loss"
        plot_training_curve(
            model.loss_history,
            title=f"{model_name} - {scenario_name} Training Curve",
            ylabel=y_label,
            filename=f"{base_name}_training_curve.png"
        )

    if hasattr(model, "train_accuracy_history") and
len(model.train_accuracy_history) > 0:
        plot_training_curve(
            model.train_accuracy_history,
            title=f"{model_name} - {scenario_name} Train Accuracy",
            ylabel="Accuracy",
            filename=f"{base_name}_train_accuracy.png"
        )
```

```
    plot_examples(
        X_test_images,
        y_test,
        y_pred,
        title=f"{model_name} - {scenario_name} | Correct
Classifications",
        correct=True,
        num_examples=10,
        filename=f"{base_name}_correct_examples.png"
    )

    plot_examples(
        X_test_images,
        y_test,
        y_pred,
        title=f"{model_name} - {scenario_name} | Incorrect
Classifications",
        correct=False,
        num_examples=10,
        filename=f"{base_name}_incorrect_examples.png"
    )

    return {
        "model": model_name,
        "scenario": scenario_name,
        "best_lr": best_lr,
        "best_epochs": best_epochs,
        "test_accuracy": acc,
        "per_class_accuracy": pca,
        "confusion_matrix": cm
    }


# ============================================================
# MAIN
# ============================================================
def main():
    print("Creating balanced and imbalanced datasets...")

    X_bal, y_bal = create_balanced_dataset(
        X_train_full, y_train_full,
        samples_per_class=BALANCED_SAMPLES_PER_CLASS
    )

    X_imb, y_imb = create_imbalanced_dataset(
        X_train_full, y_train_full,
        main_class=IMBALANCED_MAIN_CLASS,
        main_samples=IMBALANCED_MAIN_SAMPLES,
        other_samples=IMBALANCED_OTHER_SAMPLES
    )

    print("\nBalanced distribution:")
    print(Counter(y_bal))

    print("\nImbalanced distribution:")
    print(Counter(y_imb))
    print(f"\nMain class in imbalanced scenario:
{IMBALANCED_MAIN_CLASS} ({class_names[IMBALANCED_MAIN_CLASS]})")

    scenarios = [
        ("Balanced", X_bal, y_bal),
```

```
        ("Imbalanced", X_imb, y_imb)
    ]

    model_names = [
        "Perceptron OVA",
        "Perceptron OVO",
        "Softmax"
    ]

    all_search_rows = []
    final_results = []

    for scenario_name, X_scenario, y_scenario in scenarios:
        X_train_split, y_train_split, X_val_split, y_val_split =
stratified_train_val_split(
            X_scenario, y_scenario, val_ratio=VALIDATION_RATIO
        )

        for model_name in model_names:
            best_lr, best_epochs, best_val_acc, search_rows =
hyperparameter_search(
                model_name=model_name,
                scenario_name=scenario_name,
                X_train=X_train_split,
                y_train=y_train_split,
                X_val=X_val_split,
                y_val=y_val_split
            )

            all_search_rows.extend(search_rows)

            print(
                f"\nBest for {model_name} | {scenario_name}: "
                f"lr={best_lr}, epochs={best_epochs},
val_acc={best_val_acc:.4f}"
            )

            # Retrain on full scenario training data with best
hyperparameters
            final_result = evaluate_best_model(
                model_name=model_name,
                scenario_name=scenario_name,
                X_train_used=X_scenario,
                y_train_used=y_scenario,
                X_test=X_test,
                y_test=y_test,
                X_test_images=X_test_images,
                best_lr=best_lr,
                best_epochs=best_epochs
            )

            final_results.append(final_result)

    # Save grid search results
    save_grid_search_results(all_search_rows,
"grid_search_results.csv")

    # Save final results CSV
    save_final_results_csv(final_results, "final_results.csv")

    # Final summary
```

```
    print("\n" + "=" * 70)
    print("FINAL SUMMARY")
    print("=" * 70)
    for res in final_results:
        print(
            f"{res['model']:15s} | {res['scenario']:10s} | "
            f"lr={res['best_lr']:<6} |
epochs={res['best_epochs']:<3d} | "
            f"Test Accuracy = {res['test_accuracy']:.4f}"
        )

    summary_path = os.path.join(RESULTS_DIR, "summary_results.txt")
    with open(summary_path, "w", encoding="utf-8") as f:
        f.write("FINAL SUMMARY\n")
        f.write("=" * 70 + "\n")
        for res in final_results:
            f.write(
                f"{res['model']:15s} | {res['scenario']:10s} | "
                f"lr={res['best_lr']:<6} |
epochs={res['best_epochs']:<3d} | "
                f"Test Accuracy = {res['test_accuracy']:.4f}\n"
            )

    print(f"\nAll results saved in folder: {RESULTS_DIR}")


if __name__ == "__main__":
    main()
```

9. Βιβλιογραφία

Η παρούσα εργασία βασίστηκε σε βιβλία και πηγές που καλύπτουν τους τομείς της αναγνώρισης προτύπων, της μηχανικής μάθησης και της επιστήμης δεδομένων.

Βιβλία

- Theodoridis, S., & Koutroumbas, K. (2009). *Αναγνώριση Προτύπων*. Εκδόσεις Πασχαλίδης.
- Theodoridis, S., Pikrakis, A., Koutroumbas, K., & Cavouras, D. (2010). *Εισαγωγή στην Αναγνώριση Προτύπων με MATLAB*. Εκδόσεις Πασχαλίδης.
- VanderPlas, J. (2016). *Εγχειρίδιο για την Επιστήμη Δεδομένων με Python*. Εκδόσεις Παπασωτηρίου.
- Géron, A. (2019). *Μηχανική Μάθηση στην Πράξη με Scikit-Learn, Keras και TensorFlow* (3η έκδοση). Εκδόσεις Παπασωτηρίου.
- Raschka, S. (2015). *Python Machine Learning*. Packt Publishing.
- Grus, J. (2015). *Επιστήμη Δεδομένων: Βασικές Αρχές και Εφαρμογές με Python*. Εκδόσεις Παπασωτηρίου.
- McKinney, W. (2017). *Ανάλυση Δεδομένων με Python*. Εκδόσεις Παπασωτηρίου.
- Guttag, J. V. (2016). *Εισαγωγή στον Υπολογισμό και τον Προγραμματισμό με Python*. Εκδόσεις Παπασωτηρίου.
- Χατζηγιαννάκης, Ν. (2019). *Η γλώσσα Python σε βάθος*. Εκδόσεις Κλειδάριθμος.

- Σαμαράς, Ν., & Τσινίδης, Κ. (2020). *Το βιβλίο της Python: Γράφοντας κώδικα*. Εκδόσεις Κριτική.

DataSet

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*.

Διαθέσιμο στο: https://github.com/zalandoresearch/fashion-mnist

Τεκμηρίωση Βιβλιοθηκών

- NumPy Documentation: https://numpy.org
- Matplotlib Documentation: https://matplotlib.org
- Seaborn Documentation: https://seaborn.pydata.org
- TensorFlow/Keras Documentation: https://www.tensorflow.org